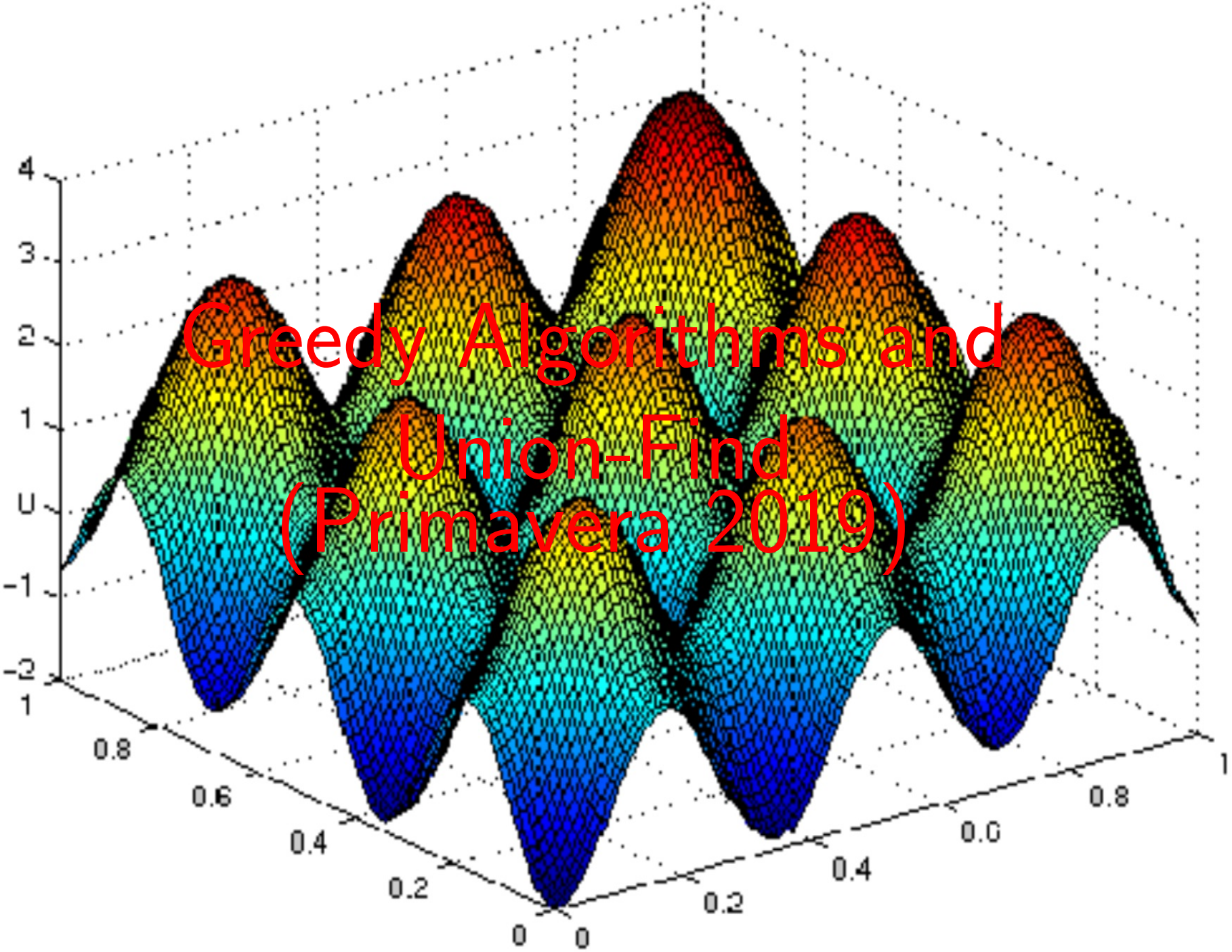
Greedy Algorithms and Union-Find (Primavera 2019)



Greedy Algorithms

A greedy algorithm, is a technique that always makes a locally optimal choice in the myopic hope that this choice will lead to a globally optimal solution.

Greedy algorithms are mainly applied to optimization problems: Given as input a set S of elements, and a function $f : S \to \mathbb{R}$, called the objective function, S we have to choose a of subset of compatible elements in S such that it maximizes (o minimizes) f.

Example: $S = G(V, E), w : E \to \mathbb{Z}$, for any $u, v \in V$, $f(u, v)$, distance between u and v. The problem consists in given specific $v, u \in V$, find the minimum graph distance between u and v. S paths of edges in G, with $f =$ sum of weights of edges between u and v.

Greedy Algorithms

Greedy algorithms are very easy to design, for most optimization problems, but they do not always yield optimal solutions. Sometimes the greedy techniques yield heuristics, not algorithms.

- At each step we choose the best (myopic) choice at the moment and then solve the subproblem that arise later.
- The choice may depend on previous choices, but not on future choices.
- At each choice, the algorithm reduces the problem into a smaller one.
- A greedy algorithm never backtracks.

Greedy Algorithms

For the greedy strategy to work correctly, it is necessary that the problem under consideration has two characteristics:

- Greedy choice property: We can arrive to the global optimum by selecting a local optimums.
- Optimal substructure: An optimal solution to the problem contains the optimal solutions to subproblems.

Fractional knapsack problem

Fractional Knapsack
INPUT:a set $I = \{i\}_1^n$ of items that can be fractioned, each i with weight w_i and value v_i. A maximum weight W permissible
QUESTION: select a set of items or fractions of item, to maximize the profit, within allowed weight W

Example.

Item	I:	1	2	3
Value	V:	60	100	120
Weight	w:	10	20	30

$W = 28$

Fractional knapsack

Greedy for fractional knapsack (I, V, W)
Sort I in decreasing value of v_i/w_i
Take the maximum amount of the first item
while Total weight taken $\leq W$ **do**
  Take the maximum amount of the next item
end while

If n is the number of items, The algorithm has a cost of $T(n) = O(n + n \log n)$.

Example.

Item	I:	1	2	3
Value	V:	60	100	120
Weight	w:	10	20	30
v/w	:	6	5	4

As $W = 28$ then take 10 of 1 and 18 of 2

Correctness?

Greedy does not always work

0-1 Knapsack

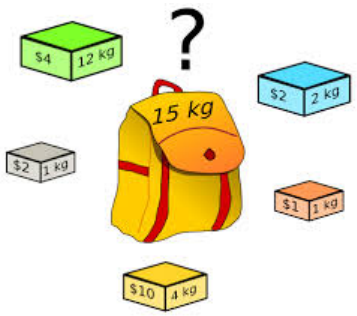
INPUT:a set I of n items that can NOT be fractioned, each i with weight w_i and value v_i. A maximum weight W permissible

QUESTION: select the items to maximize the profit, within allowed weight W.

For example

Item	I:	1	2	3
Value	V:	60	100	120
Weight	w:	10	20	30
v/w	:	6	5	4

with



$W = 50$.

Then any solution which includes item 1 is not optimal. The optimal solution consists of items 2 and 3.

Activity scheduling problems

A set of activities $S = \{1, 2, \ldots, n\}$ to be processed by a single processor, according to different constrains.

1. Interval scheduling problem: Each $i \in S$ has a start time s_i and a finish time f_i. Maximize the set of mutually compatible activities
2. Weighted interval scheduling problem: Each $i \in S$ has a s_i, a f_i, and a weight w_i. Find the set of mutually compatible such that it maximizes $\sum_{i \in S} w_i$
3. Job scheduling problem (Lateness minimization): Each $i \in S$ has a processing time t_i (could start at any time s_i) but it has a deadline d_i, define lateness L_i of i by $\max_i\{0, (s_i + t_i) - d_i\}$. Find the schedule of the s_i for all the tasks s.t. no two tasks are planned to be processed at the time and the lateness is minimized.

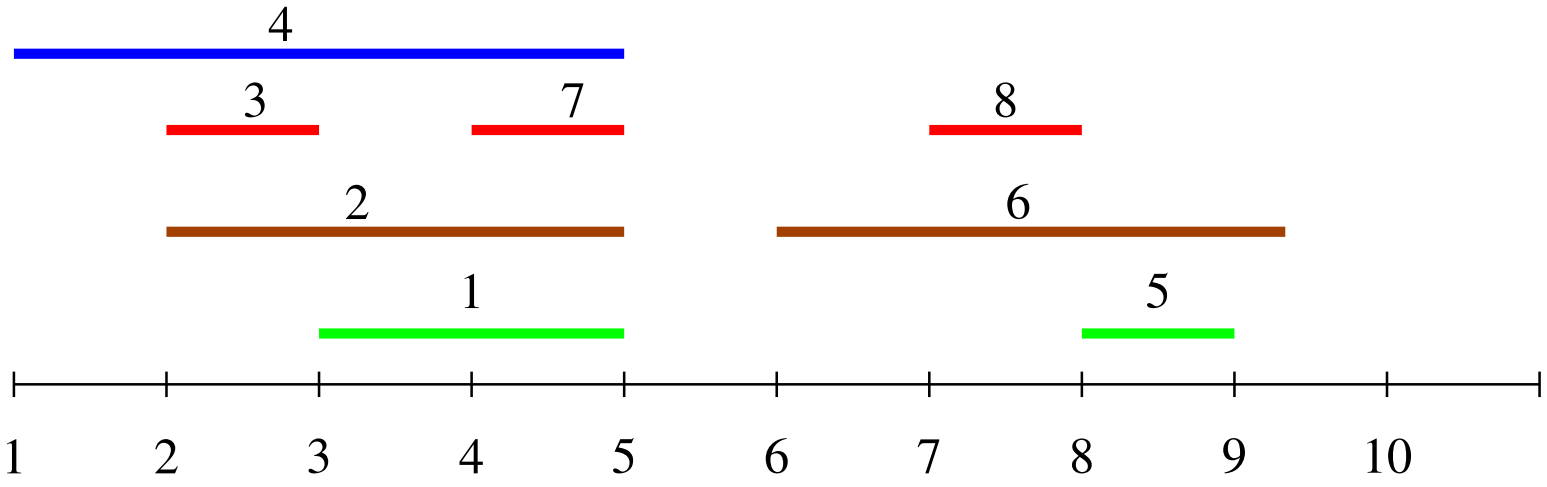
The interval scheduling problem

Activity Selection Problem
INPUT: a set $S = \{1, 2, \ldots, n\}$ of activities to be processed by a single resource. Each activity i has a start time s_i and a finish time f_i, with $f_i > s_i$.
QUESTION: Maximize the set of mutually compatible activities, where activities i and j are compatible if $[s_i f_i) \cap (s_j f_j] = \emptyset$.

Notice, the set of compatible solution is the set of activities with empty intersection, and the objective function to maximize is the cardinality of every compatible set.

Example.

Activity :	1	2	3	4	5	6	7	8
Start (s):	3	2	2	1	8	6	4	7
Finish (f):	5	5	3	5	9	9	5	8



To apply the greedy technique to a problem, we must take into consideration the following,

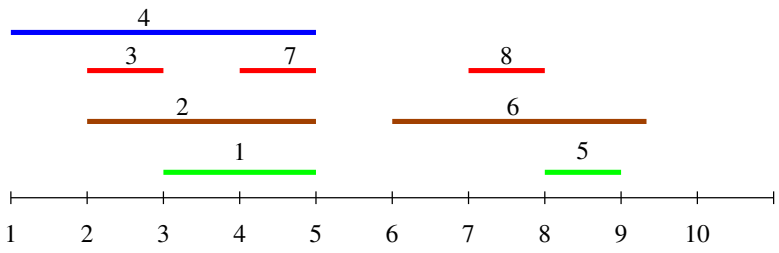
- A local criteria to allow the selection,
- a condition to determine if a partial solution can be completed,
- a procedure to test that we have the optimal solution.

The Activity Selection problem.

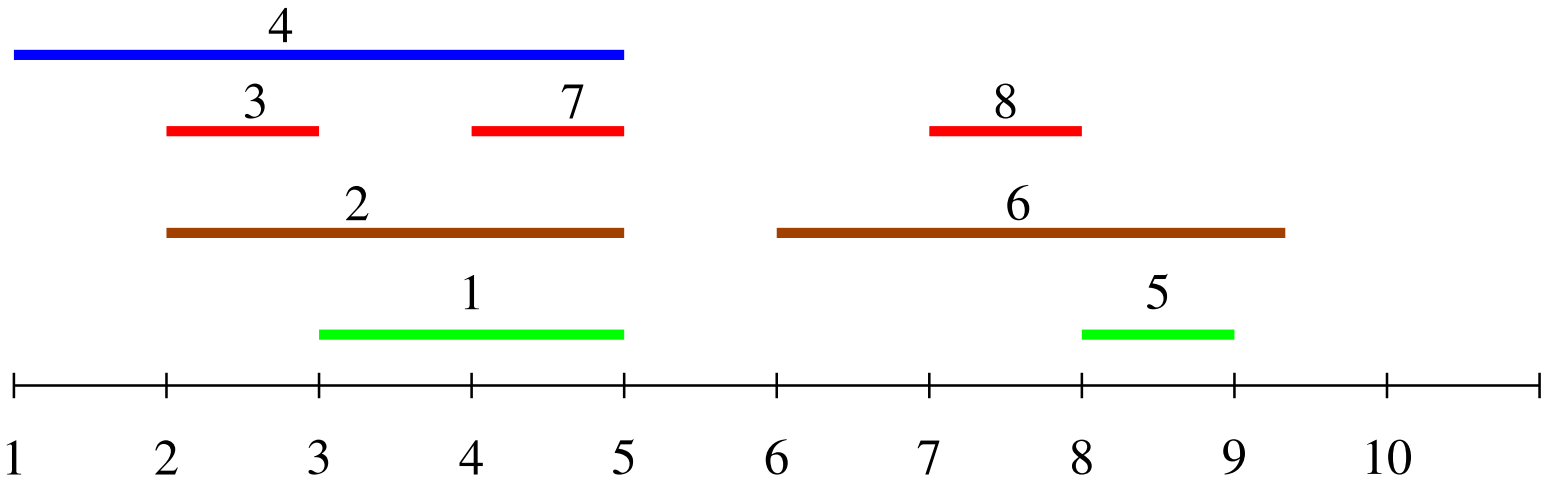
Given a set A of activities, wish to maximize the number of compatible activities.

Activity selection A
Sort A by increasing order of f_i
Let $a_1, a_2, \ldots, a_n$ the resulting sorted list of activities
$S := \{a_1\}$
$j := 1$ {pointer in sorted list}
for $i = 2$ **to** n **do**
  if $s_i \geq f_j$ **then**
    $S := S \cup \{a_i\}$ and $j := i$
  end if
end for
return S.

A : 3 1 2 7 8 5 6; f_i : 3 5 5 5 8 9 9
$\Rightarrow$ SOL: 3 1 8 5



Notice: In the activity problem we are maximizing the number of activities, independently of the occupancy of the resource under consideration. For example in:



solution 3185 is as valid as 3785. If we were asking for maximum occupancy 456 will be a solution.

Problem: How would you modify the previous algorithm to deal with the maximum occupancy problem?

Theorem

The previous algorithm produces an optimal solution to the activity selection problem.

There is an optimal solution that includes the activity with earlier finishing time.

Proof.

Given $A = \{1, \ldots, n\}$ sorted by finishing time, we must show there is an optimal solution that begins with activity 1. Let $S = \{k, \ldots, m\}$ be a solution. If $k = 1$ done. Otherwise, define $B = (S - \{k\}) \cup \{1\}$. As $f_1 \leq f_k$ the activities in B are disjoint. As $|B| = |S|$, B is also an optimal solution. If S is an optimal solution to A, then $S' = A - \{1\}$ is an optimal solution for $A' = \{i \in A | s_i \geq f_1\}$. Therefore, after each greedy choice we are left with an optimization problem of the same form as the original. Induction on the number of choices, the greedy strategy produces an optimal solution $\square$

Notice the optimal substructure of the problem: If an optimal solution S to a problem includes a_k, then the partial solutions excluding a_k from S should also be optimal in their corresponding domains.

Greedy does not always work.

Weighted Activity Selection Problem

INPUT: a set $A = \{1, 2, \ldots, n\}$ of activities to be processed by a single resource. Each activity i has a start time s_i and a finish time f_i, with $f_i > s_i$, and a weight w_i.

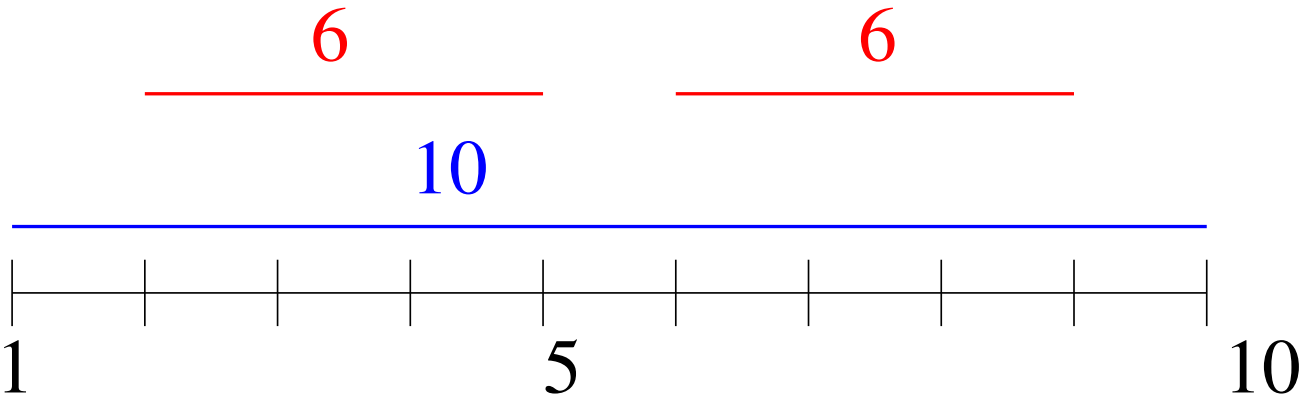
QUESTION: Find the set of mutually compatible activities such that it maximizes $\sum_{i \in S} w_i$

$S := \emptyset$
sort $W = \{w_i\}$ by decreasing value
choose the max. weight w_m
add $m = (s_m, f_m)$ to S
remove all w from W,
which the correspond to activities overlapping with m
while there are $w \in W$ **do**
  repeat the greedy procedure
end while
return S

Correctness?

Greedy does not always work.

The previous greedy does not always solve the problem!



The algorithm chooses the interval $(1, 10)$ with weight 10, and the solution is the intervals $(2, 5)$ and $(5, 9)$ with total weight of 12

Job scheduling problem

Also known as the Lateness minimisation problem.
We have a single resource and n requests to use the resource, each request i taking a time t_i.

In contrast to the previous problem, each request instead of having an starting and finishing time, it has a deadline d_i. The goal is to schedule the resources (processors) as to minimize over all the requests, the maximal amount of time that a request exceeds the deadline.

Minimize Lateness

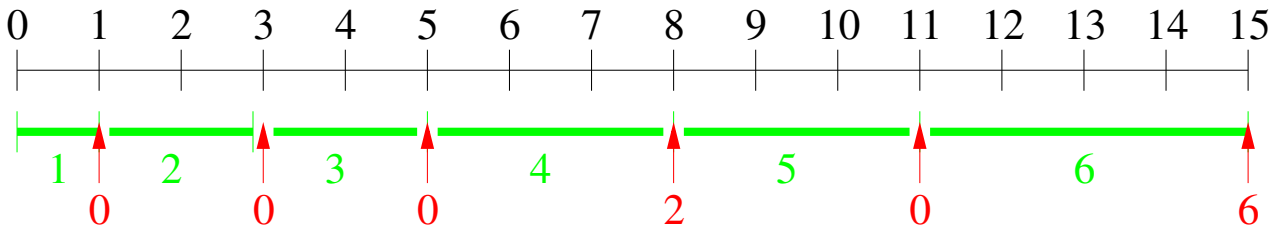
- We have a single processor
- We have n jobs such that job i:
  - requires t_i units of processing time,
  - it has to be finished by time d_i,
- Lateness of i:

$$L_i = \begin{cases} 0 & \text{if } f_i \leq d_i, \\ f_i - d_i & \text{otherwise.} \end{cases}$$

i	t_i	d_i
1	1	9
2	2	8
3	2	15
4	3	6
5	3	14
6	4	9

Goal: schedule the jobs to minimize the maximal lateness, over all the jobs

i.e. We must assign starting time s_i to each i, as to $\min_i \max L_i$.



Minimize Lateness

Schedule jobs according to some ordering

(1.-) Sort in increasing order of t_i:
Process jobs with short time first

i	t_i	d_i
1	1	6
2	5	5

(2.-) Sort in increasing order of $d_i - t_i$:
Process first jobs with less slack time

i	t_i	d_i	$d_1 - t_i$
1	1	2	1
2	10	10	0

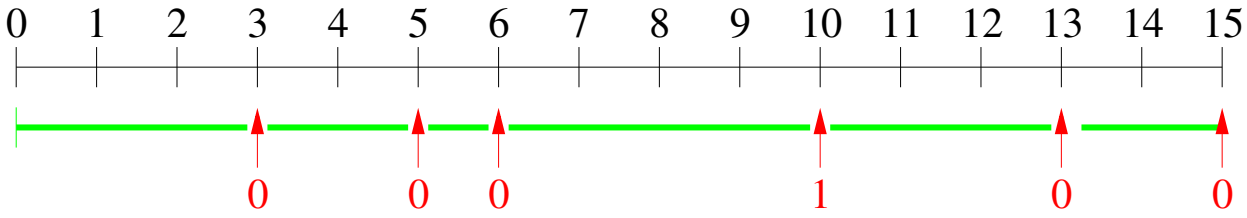
In this case, job 2 should be processed first, which doesn't minimise lateness.

Process urgent jobs first

(3.-) Sort in increasing order of d_i.

LatenessA $\{i, t_i, d_i\}$
SORT by increasing order of d_i:
$\{d_1, d_2, \ldots, d_n\}$
Rearrange the jobs $i : 1, 2, \ldots, n$
$t = 0$
for $i = 2$ **to** n **do**
  Assign job i to $[t, t + t_i]$
  $t = t + t_i$
  $s_i = t; f_i = t + t_i$
end for
return $S = \{[s_1, f_1], \ldots [s_n, f_n]\}$.

i	t_i	d_i	sorted i
1	1	9	3
2	2	8	2
3	2	15	6
4	3	6	1
5	3	14	5
6	4	9	4



d: 6 8 9 9 14 15
i: 1 2 3 4 5 6

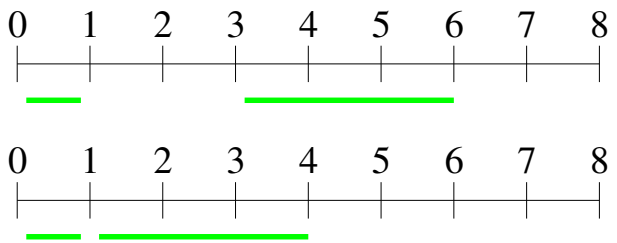
Complexity and idle time

Time complexity

Running-time of the algorithm without comparison sorting: $O(n)$
Total running-time: $O(n \lg n)$

Idle steps

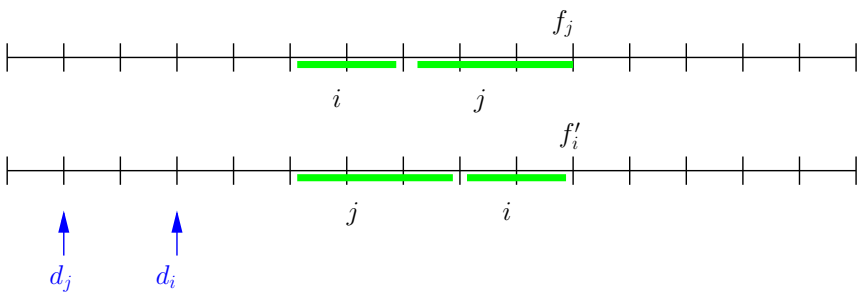
From an optimal schedule with idle steps, we always can eliminate gaps to obtain another optimal schedule:



There exists an optimal schedule with no idle steps.

Inversions and exchange argument

An schedule has an inversion if i is scheduled before j with $d_j < d_i$.



Lemma

Exchanging two inverted jobs reduces the number of inversions by 1 and does not increase the max lateness.

Proof Let $L =$ lateness before exchange and let L' be the lateness after the exchange, let L_i, L_j, L'_i, L'_j, the corresponding quantities for i, j.

Notice that $f_j > f_i$, $f'_j < f'_j$ and $f'_j < f_i$, using the fact that $d_j < d_i$
$\Rightarrow L'_i = f'_i - d_i < f_j - d_i < f_j - d_j = L_j$

Therefore the swapping does not increase the maximum lateness of the schedule. $\square$

Correctness of LatenessA

Notice the output S produced by LatenessA has no inversions and no idle steps.

Theorem

Algorithm LatenessA returns an optimal schedule S.

Proof

Assume $\hat{S}$ is an optimal schedule with the minimal number of inversions (and no idle steps).
If $\hat{S}$ has 0 inversions then $\hat{S} = S$.
If number inversions in $\hat{S}$ is > 0, let $i - j$ be an adjacent inversion.
Exchanging i and j does not increase lateness and decrease the number of inversions.
Therefore, max lateness $S \leq$ max lateness $\hat{S}$. $\square$

Network construction: Minimum Spanning Tree

- We have a set of locations $V = \{v_1, \ldots, v_n\}$,
- we want to build a communication network on top of them
- we want that any v_i can communicate with any v_j,
- for any pair (v_i, v_j) there is a cost $w(v_i, v_j)$ of building a direct link,
- if E is the set of all possible edges ($|E| \leq n(n-1)/2$), we want to find a subset $T(E) \subseteq E$ s.t. $(V, T(E))$ is connected and minimizes $\sum_{e \in T(E)} w(e)$.

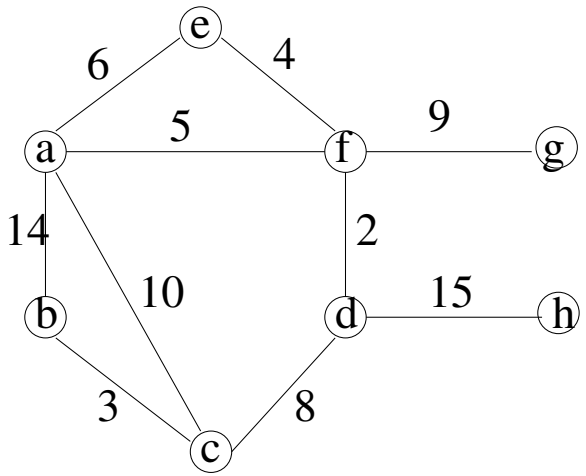
Network construction: Minimum Spanning Tree

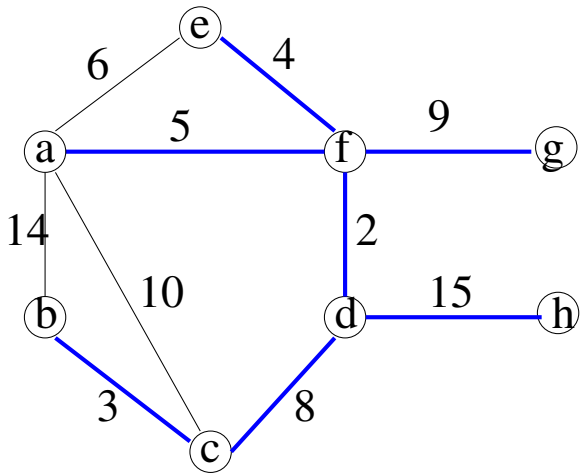
- We have a set of locations $V = \{v_1, \ldots, v_n\}$,
- we want to build a communication network on top of them
- we want that any v_i can communicate with any v_j,
- for any pair (v_i, v_j) there is a cost $w(v_i, v_j)$ of building a direct link,
- if E is the set of all possible edges ($|E| \leq n(n-1)/2$), we want to find a subset $T(E) \subseteq E$ s.t. $(V, T(E))$ is connected and minimizes $\sum_{e \in T(E)} w(e)$.

Construct the MST

Minimum Spanning Tree (MST)

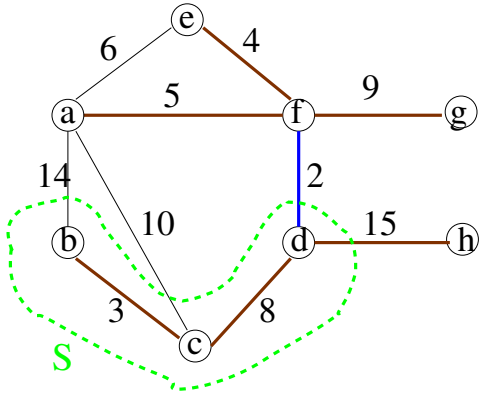
INPUT: An edge weighted graph $G = (V, E)$,
$|V| = n, \forall e \in E, w(e) \in \mathbb{R}$,
QUESTION: Find a tree T with $V(T) = V$ and $E(T) \subseteq E$, such that it minimizes $w(T) = \sum_{e \in E(T)} w(e)$.





Some definitions

Given $G = (V, E)$:
A path is a sequence of consecutive edges. A cyle is a path with no repeated vertices other that the one that it starts and ends.
A cut is a partition of V into S and $V - S$.
The cut-set of a cut is the set of edges with one end in S and the other in $V - S$.



Overall strategy

Given a MST T in G, with different edge weights, T has the following properties:

- Cut property
  $e \in T \Leftrightarrow e$ is the lighted edge across some cut in G.
- Cycle property
  $e \notin T \Leftrightarrow e$ is the heaviest edge on some cycle in G.

The MST algorithms are methods for ruling edges in or out of T.

The $\Leftarrow$ implication of the cut property will yield the blue (include) rule, which allow us to include a min weight edge in T for $\exists$ cut.

The $\Rightarrow$ implication of the cycle property will yield the red (exclude) rule which allow us to exclude a max weight edge from T for $\exists$ cycles.

The cut rule (Blue rule)

Given an optimal MST T, removing an edge e yields T_1 and T_2 which are optimal for each subgraph.

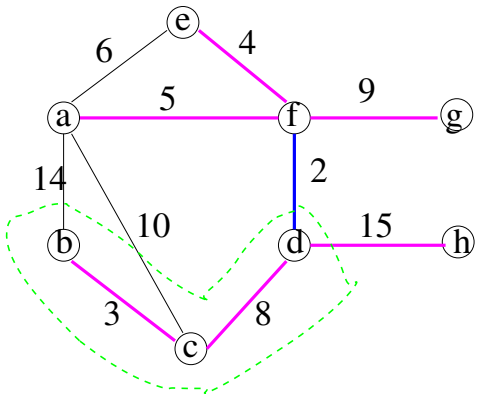
Theorem (The cut rule)

Given $G = (V, E), w : E \to \mathbb{R}^+$, let T be a MST of G and $S \subseteq T$. Let $e = (u, v)$ an min-weight edge in G connecting S to $V - S$. Then $e \in T$.

The edges incorporated to the solution by this rule, are said to be blue.

Proof.

Assume $e \notin T$. Consider a path from u to v in T. Replacing the first edge in the path, which is not in S, by e, must give a spanning tree of equal or less weight. $\square$



The cycle rule (Red rule)

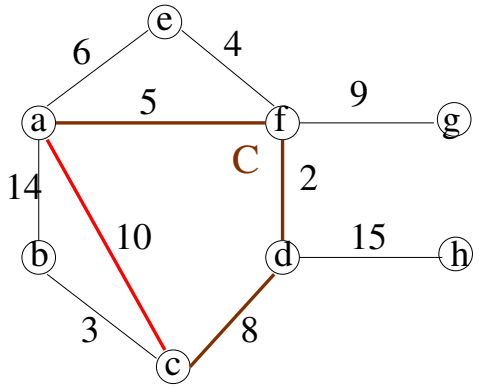
Theorem (The cycle rule)

Given $G = (V, E), w : E \to \mathbb{R}^+$, let C be a cycle in G, the edge $e \in C$ with greater weight can not be part of the optimal MST T.

The edges processed by this rule, are said to be red.

Proof.

Let C be a cycle spanning through vertices $\{v_i, \ldots, v_l\}$, then removing the max weighted edge gives a a better solution. $\square$



C=cycle spanning {a,c,d,f}

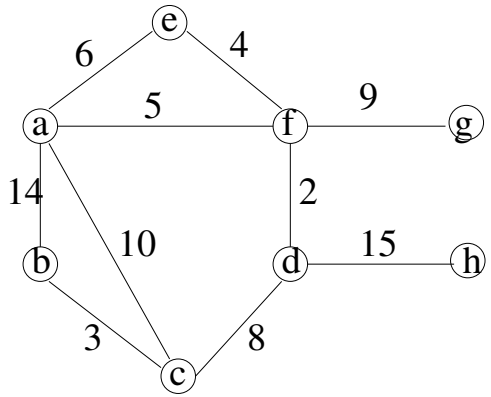
This doesn't mean that the max. weighted edge would not be in the MST.

Generic greedy for MST: Apply blue and/or red rules

The Min. Spanning Tree problem has the optimal substructure problem we can apply greedy.
Robert Tarjan: Data Structures and Network Algorithms, SIAM , 1984

Blue rule: Given a cut-set between S and $V - S$ with no blue edges, select from the cut-set a non-colored edge with min weight and paint it blue

Red rule: Given a cycle C with no red edges, selected an non-colored edge in C with max weight and paint it red.



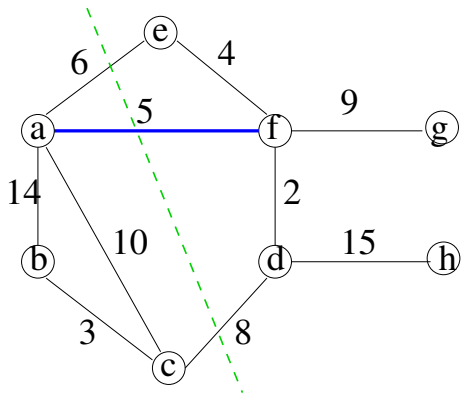
Greedy scheme:
Given G, $V(G) = n$, apply the red and blue rules until having $n - 1$ blue edges, those form the MST.

Greedy for MST

The Min. Spanning Tree problem has the optimal substructure problem we can apply greedy.
Robert Tarjan: Data Structures and Network Algorithms, SIAM , 1984

Blue rule: Given a cut-set between S and $V - S$ with no blue edges, select a non-colored edge with min weight and paint it blue

Red rule: Given cycle C with no red edges, selected a non-colored edge in C with max weight and paint it red.



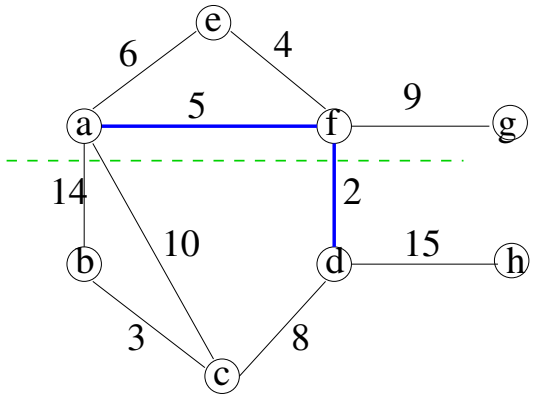
Greedy scheme:
Given G, $V(G) = n$, apply the red and blue rules until having $n - 1$ blue edges, those form the MST.

Greedy for MST

The Min. Spanning Tree problem has the optimal substructure problem we can apply greedy.
Robert Tarjan: Data Structures and Network Algorithms, SIAM , 1984

Blue rule: Given a cut-set between S and $V - S$ with no blue edges, select a non-colored edge with min weight and paint it blue

Red rule: Given cycle C with no red edges, selected a non-colored edge in C with max weight and paint it red.



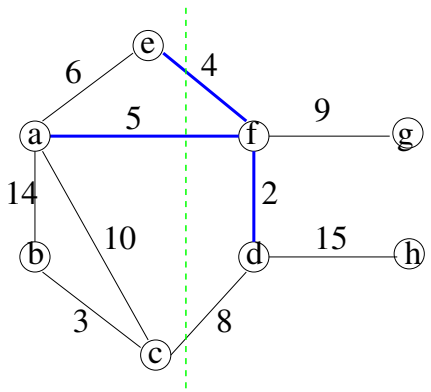
Greedy scheme:
Given G, $V(G) = n$, apply the red and blue rules until having $n - 1$ blue edges, those form the MST.

Greedy for MST

The Min. Spanning Tree problem has the optimal substructure problem we can apply greedy.
Robert Tarjan: Data Structures and Network Algorithms, SIAM , 1984

Blue rule: Given a cut-set between S and $V - S$ with no blue edges, select a non-colored edge with min weight and paint it blue

Red rule: Given cycle C with no red edges, selected a non-colored edge in C with max weight and paint it red.



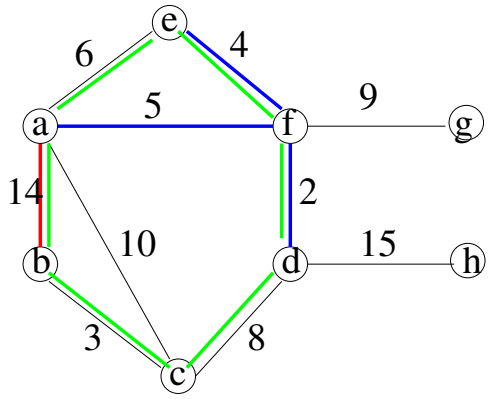
Greedy scheme:
Given G, $V(G) = n$, apply the red and blue rules until having $n - 1$ blue edges, those form the MST.

Greedy for MST

The Min. Spanning Tree problem has the optimal substructure problem we can apply greedy.
Robert Tarjan: Data Structures and Network Algorithms, SIAM , 1984

Blue rule: Given a cut-set between S and $V - S$ with no blue edges, select a non-colored edge with min weight and paint it blue

Red rule: Given cycle C with no red edges, selected a non-colored edge in C with max weight and remove it.



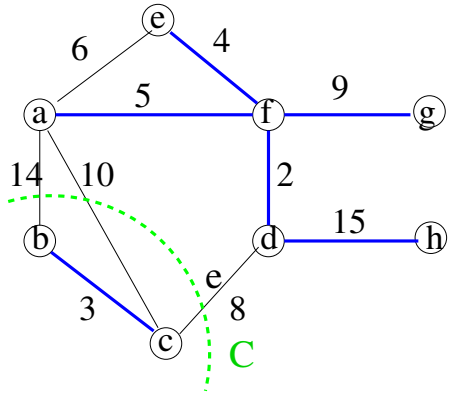
Greedy scheme:
Given G, $V(G) = n$, apply the red and blue rules until having $n - 1$ blue edges, those form the MST.

Greedy for MST : Correctness

Theorem

There exists a MST T containing only all blue edges. Moreover the algorithm finishes and finds a MST

Sketch of proof Induction on number of iterations for blue and red rules. The base case (no edges colored) is trivial. The induction step is the same that in the proofs of the cut and cycle rules. Moreover if we have an e not colored, if e ends are in different blue trees, apply blue rule, otherwise color red e. □



We need implementations for the algorithm! The ones we present use only the blue rule

A short history of MST implementation

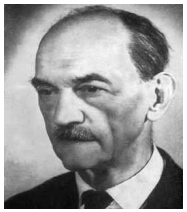
There has been extensive work to obtain the most efficient algorithm to find a MST in a given graph:

- O. Borůvka gave the first greedy algorithm for the MST in 1926. V. Jarnik gave a different greedy for MST in 1930, which was re-discovered by R. Prim in 1957. In 1956 J. Kruskal gave a different greedy algorithms for the MST. All those algorithms run in $O(m \lg n)$.
- Fredman and Tarjan (1984) gave a $O(m \log^* n)$ algorithm, introducing a new data structure for priority queues, the Fibbonacci heap. Recall $\log^* n$ is the number of operations to go from $n = 1$ to $\log^* 1000 = 4$.
- Gabow, Galil, Spencer and Tarjan (1986) improved Fredman-Tarjan to $O(m \log(\log^* n))$.
- Karger, Klein and Tarjan (1995) $O(m)$ randomized algorithm.
- In 1997 B. Chazelle gave an $O(m\alpha(n))$ algorithm, where $\alpha(n)$ is a very slowly growing function, the inverse of the Ackermann function.

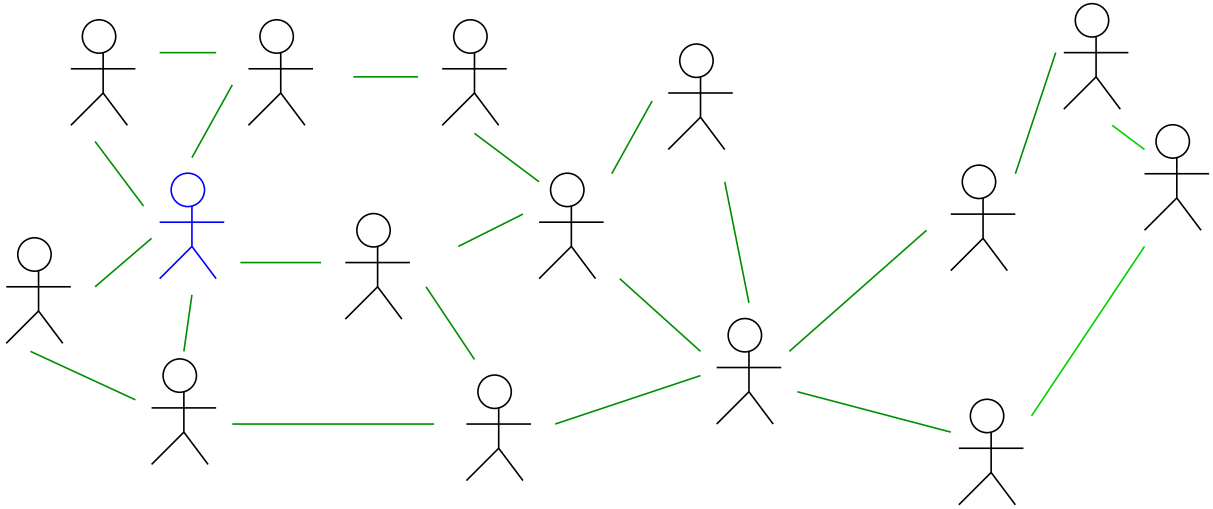
Basic algorithms

Use the greedy

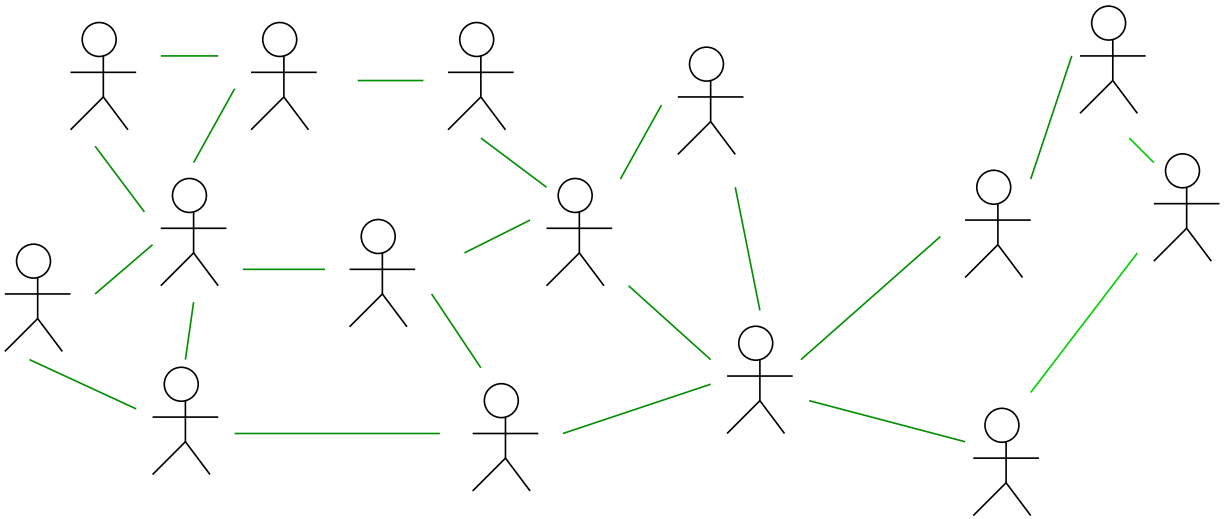
- Jarnik-Prim (Serial centralized) Starting from a vertex v, grows T adding each time the lighter edge already connected to a vertex in T, using the blue's rule. Uses a priority queue (usually a heap) to store the edges to be added and retrieve the lighter one.
- Kruskal (Serial distributed) Considers every edge and grows a forest F by using the blue and red rules to include or discard e. The insight of the algorithm is to consider the edges in order of increasing weight. This makes the complexity of Kruskal's to be dominated by $\Omega(m \lg m)$. At the end F becomes T. The efficient implementation of the algorithm uses Union-find data structure.



Jarnik-Prim vs. Kruskal

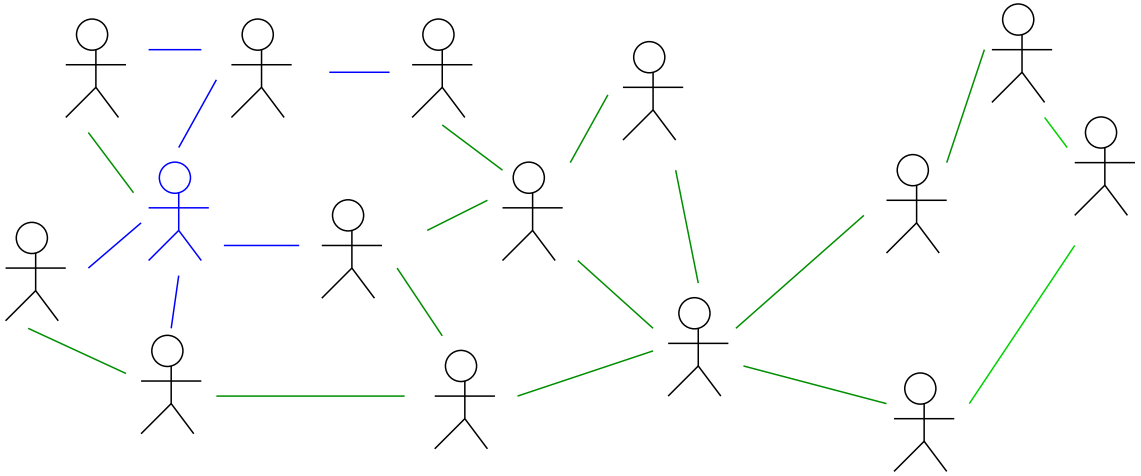


Jarnik–Prim: How blue man can spread his message to everybody

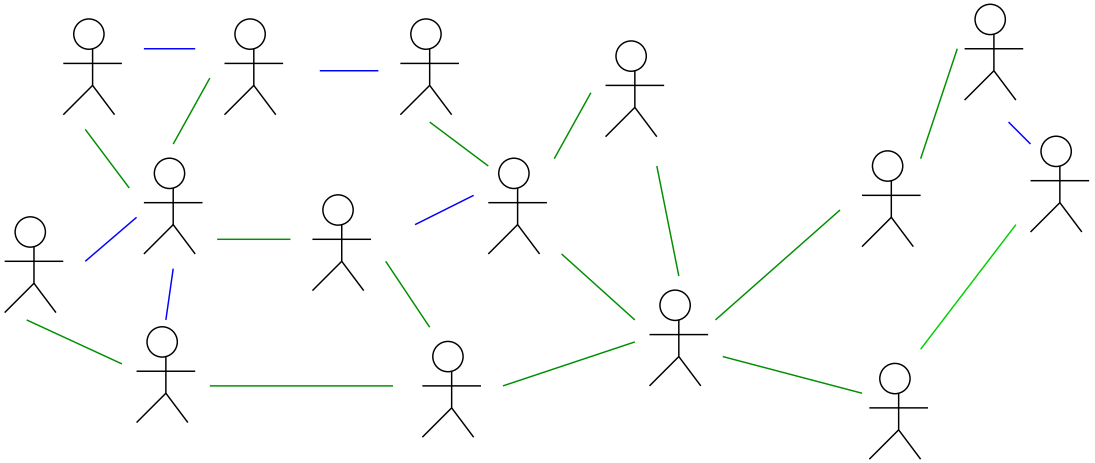


Kruskal: How to stablish a min distance cost network about all men

Jarník-Prim vs. Kruskal



Jarnik–Prim: How blue man can spread his message to everybody
(first 6 steps)



Kruskal: How to stablish a min distance cost network about all men

(6 first edges)

Jarník - Prim greedy algorithm.

V. Jarník, 1936, R. Prim, 1957

Greedy on vertices with Priority queue

Starting with an arbitrary node r, at each step build the MST by incrementing the tree with an edge of minimum weight, which does not form a cycle.

MST (G, w, r)
$T := \emptyset$
for $i = 1$ **to** $|V|$ **do**
  Let $e \in E$: e touches T, it has min weight, and do not form a cycle
  $T := T \cup \{e\}$
end for

Use a priority queue to choose min e connected to the tree already formed.

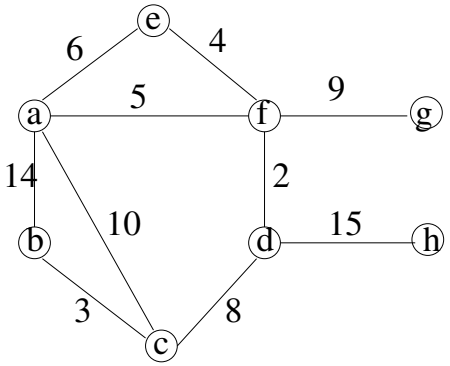
For every $v \in V - T$, let $k[v] =$ minimum weight of any edge connecting v to any vertex in T.

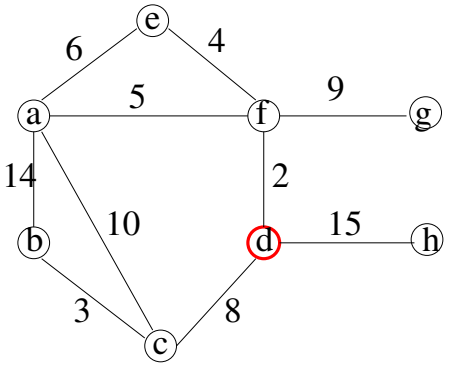
Start with $k[v] = \infty$ for all v.

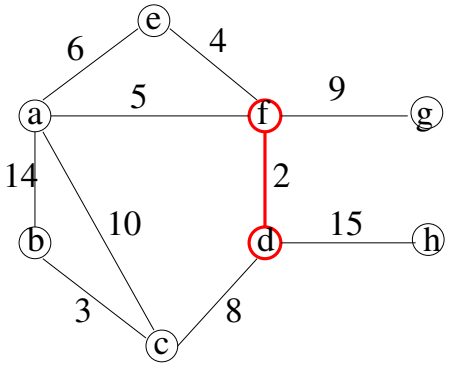
For $v \in T$, let $\pi[v]$ be the parent of v. During the algorithm $T = \{(v, \pi[v]) : v \in V - \{r\} - Q\}$

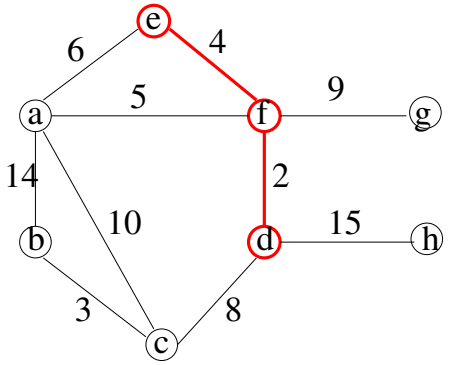
where r is the arbitrary starting vertex and Q is a min priority queue storing $k[v]$. The algorithm finishes when $Q = \emptyset$.

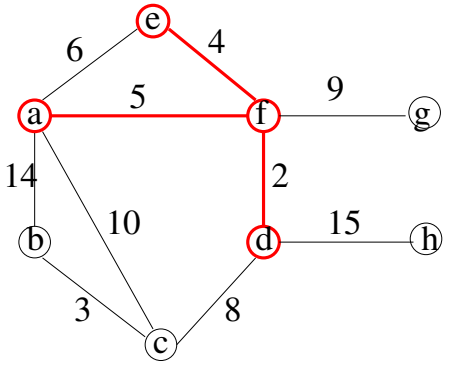
Example.

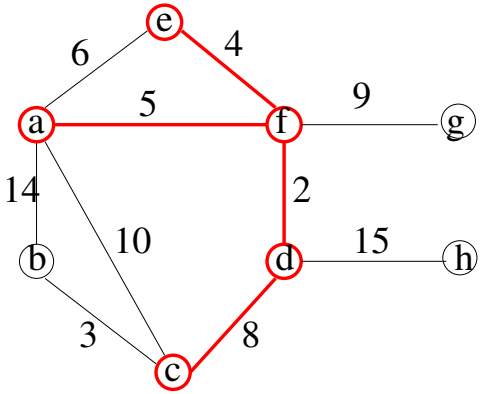

e
6
4
a
5
f
9
g
14
2
10
b
d
15
h
3
8
c

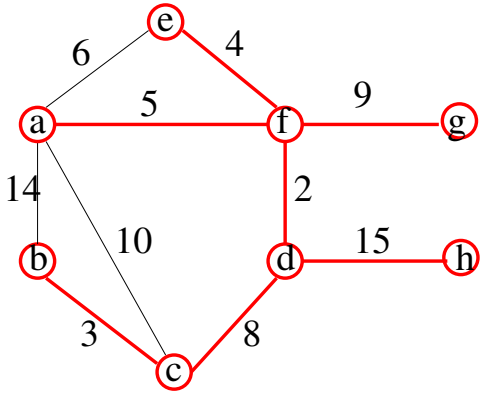

e
6
4
a
5
f
9
g
14
2
10
b
d
15
h
3
8
c

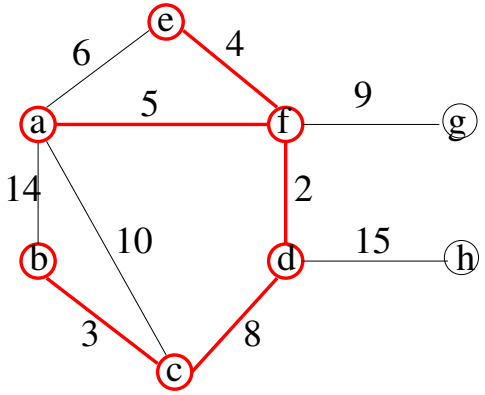

e
6
4
a
5
f
9
g
14
2
10
b
d
15
h
3
8
c


e
6
4
a
5
f
9
g
14
2
10
b
d
15
h
3
8
c


e
6
4
a
5
f
9
g
14
2
10
b
d
15
h
3
8
c


e
6
4
a
5
f
9
g
14
2
10
b
d
15
h
3
8
c





$$w(T) = 52$$

Time: depends on the implementation of Q.

Q an unsorted array: $T(n) = O(|V|^2)$;
Q a heap: $T(n) = O(|E| \lg |V|)$.
Q a Fibonacci heap: $T(n) = O(|E| + |V| \lg |V|)$

Kruskal's greedy algorithm.

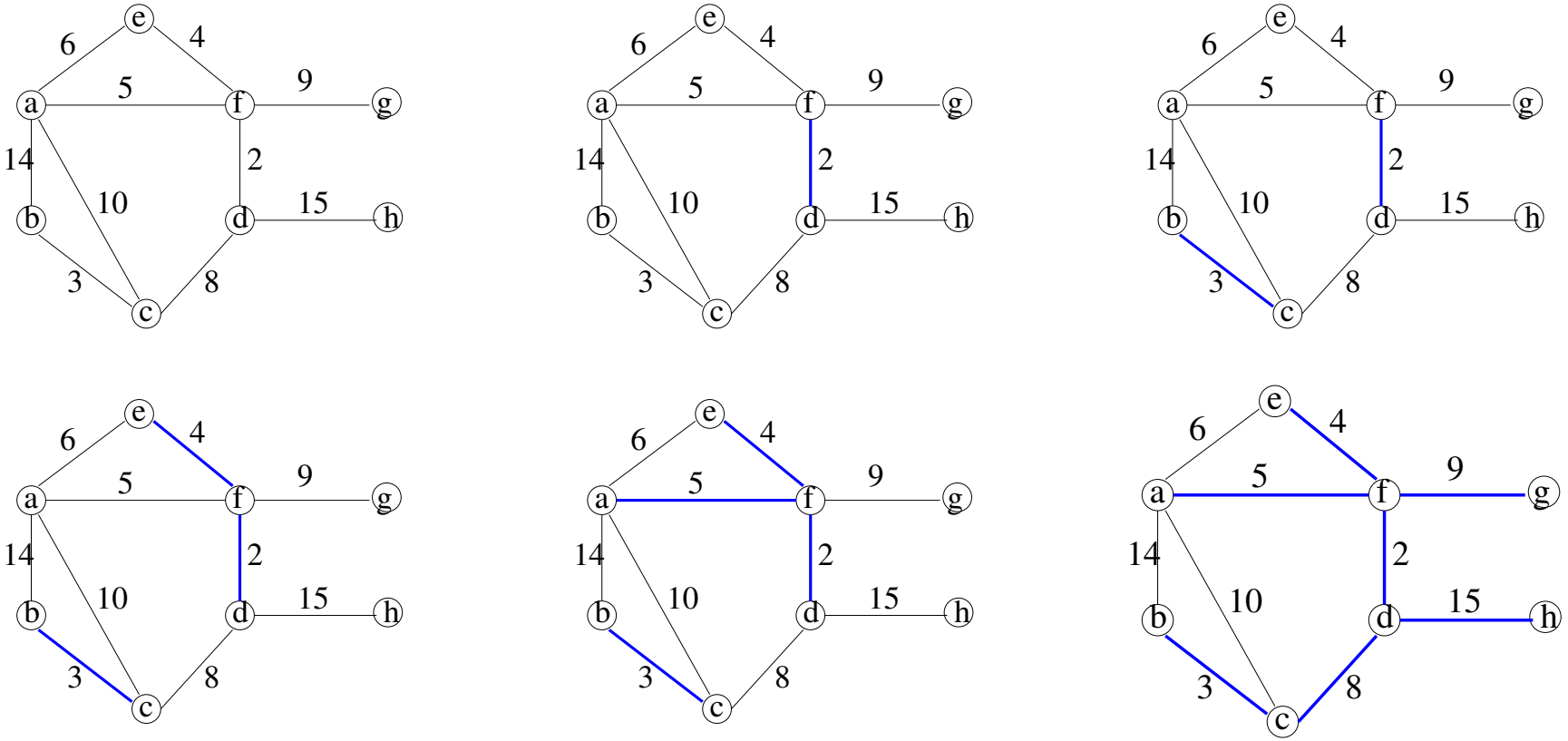
J. Kruskal, 1956

Similar to Jarník - Prim, but chooses minimum weight edges, without keeping the graph connected.

MST (G, w, r)
Sort E by increasing weight
$T := \emptyset$
for $i = 1$ **to** $|V|$ **do**
  Let $e \in E$: with minimum weight and do not form a cycle with T
  $T := T \cup \{e\}$
end for

We have an $O(m \lg m)$ from the sorting the edges.
But as $m \leq n^2$ then $O(m \lg m) = O(m \lg n)$.

Example.



Disjoint Set Union-Find

B. Galler, M. Fisher: An improved equivalence algorithm. ACM Comm., 1964; R.Tarjan 1979-1985

- Union-Find is a data structure to maintain any collection of dynamic disjoint sets.
- Union-Find is one of the most elegant data structures in the algorithmic toolkit.
- Union-Find makes possible to design almost linear time algorithms for problems that otherwise would be unfeasible.
- Union-Find is a first introduction to an active research field in algorithmic; Self organizing data structures.

Some applications of Union-Find

- Kruskal's algorithm for MST.
- Dynamic graph connectivity in very large networks.
- Cycle detection in undirected graph.
- Random maze generation and exploration.
- Percolation.
- Strategies for games: Hex and Go.
- Least common ancestor.
- Compiling equivalence statements.
- Equivalence of finite state automata.

Partition and equivalent relations

Remember a **partition** of an n element set S is collection $\{S_1, \ldots, S_k\}$ of subsets s.t.:

$$\forall S_i \subseteq S; \cup_{i=1}^{k} S_i = S; \forall S_i, S_j \text{ then } S_i \cap S_j = \emptyset$$

.

Recall also that a partition implies an equivalence relation:

$$\forall x, y \in S, x \equiv y \text{ iff } x \in S_i \& y \in S_i.$$

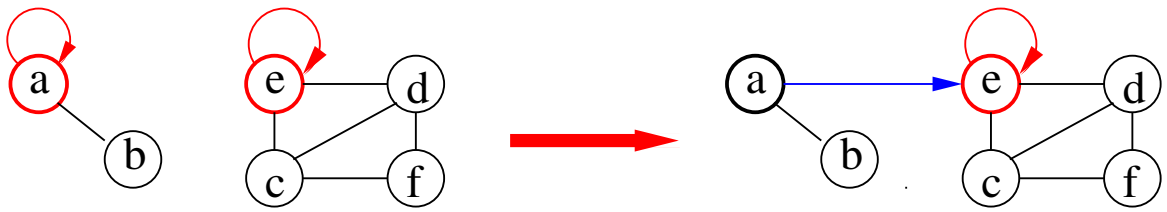
The collection $\{S_1, \ldots, S_k\}$ are the equivalence classes of the equivalence relation.

Union-Find

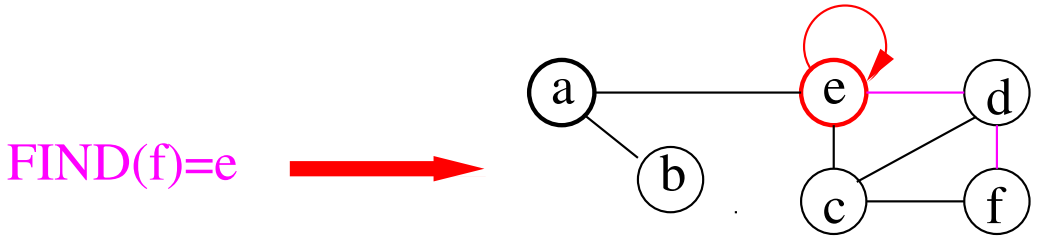
Union-Find is a data structure that supports three operations on partitions of a set:

MAKESET (x): creates a new set containing the single element x.

UNION (x, y): Merge the sets containing x and y, by using their union.



FIND (x): Return the representative of the set containing x.



Warning about UNION operation

Warning: For any $x, y \in S$ we can do UNION(x, y), x, y do not need to be representatives, but from the point of view to study the complexity of the different implementations, and separate the complexity of the operations UNION and FIND, we would consider that if x and/or y are not representatives then

$$\text{UNION } (x, y) = \text{UNION } (\text{FIND}(x), \text{FIND}(y)).$$

Union-Find Data Structure: The basic working

Given a set S of size n, construct a data structure that maintains a collection $\{S_1, \ldots, S_k\}$ of disjoint dynamic sets, each set identified by a *representative*,.

We have n initial elements a set S, we start by applying n times MAKESET to have n single element sets.

After, we want to implement a sequence of m UNION and FIND operations on the initial sets, using the minimum number of steps.

Union find is a nice data structure, for dynamic settings, where membership evolves with time.

Graph connectivity

- Consider friendship network, which is a social network on S people, where each person is represented by a node x, and the set of (undirected edges) is defined: $x, y \in S$, $(x, y) \in E$ if x and y are friend Those networks could be very large.
- Let $G = (S, E)$ represent the friendship network.
- An interested problem is social modeling is identifying connected components in $G = (S, E)$.
- A connected component is a maximal set of vertices that are connected.
- That is, a connected component is a $C \subset S$ such that:
  - For every $x, y \in C$ either $(x, y) \in E(C)$ or there is a path $x \rightsquigarrow y$.
  - No $x \in C$ has friends outside of C.

Testing if two elements are in the same connected component (FIND)

Given $G = (S, E)$ decomposed in their connected components, $C_1, \ldots, C_n$, and two $x, y \in S$.

SAME-COMPO (u, v)
if FIND (u) = FIND (v) **then**
  return true
else
  return false
end if

Find the larger connecter components in a graph: (UNION)

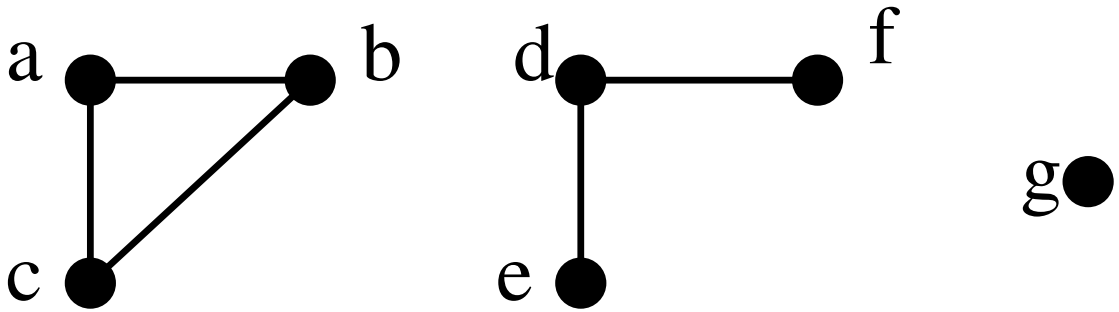
Given a set S of n people with a set $E = \{(x, y)\}$ of m pairs of friends from S, let $G = (S, E)$ be the friendship network.
The output is for each person $x \in S$, the representative of the connected component to which x belongs.

CONNECTED-COMP. $G = (S, E)$
for each $x \in S$ **do**
  MAKESET (x)
end for
for each $(x, y) \in E$ **do**
  if FIND $(x) \neq$ FIND (y) **then**
    UNION (x, y)
  end if
end for

We will see that using Union-Find, we would implement the algorithm in effectively linear time.

Graph connectivity: Example

Given $G = (S, E)$ with $S = \{a, b, c, d, e, f, g\}$, and $E = \{(a, b), (a, c), (c, b)\}$,



$\{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{f\}, \{g\},$

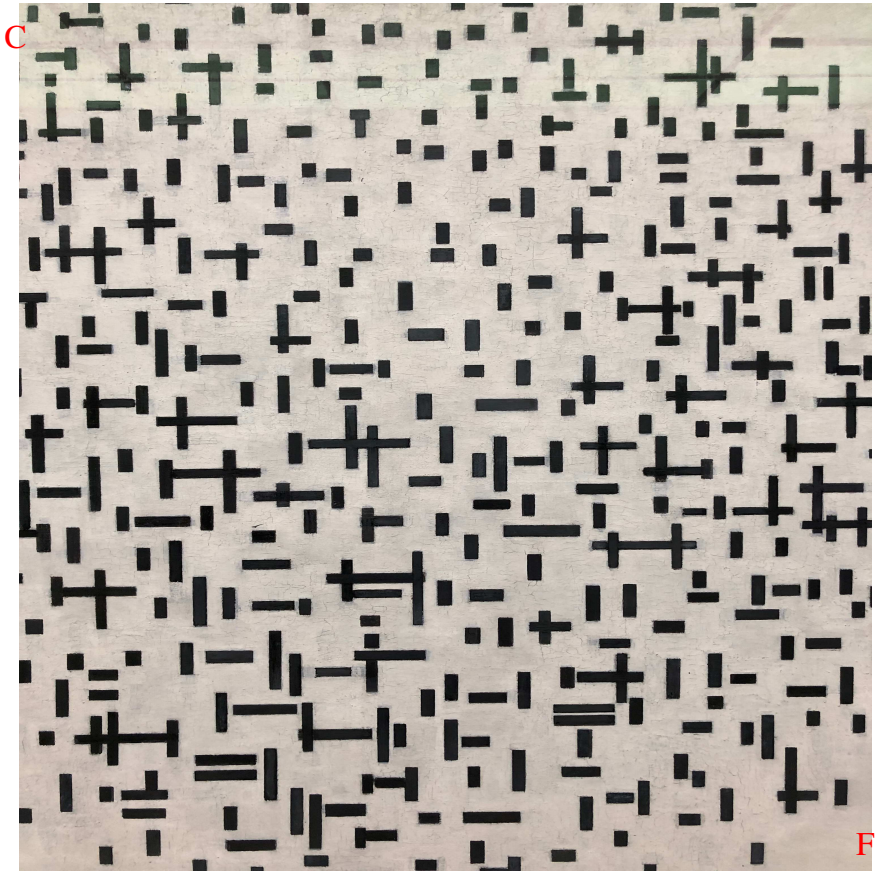
$\{a\}, \{bc\}, \{e, d\}, \{f\}, \{g\},$

$\{a\ b\ c\}, \{e\ d\ f\}, \{g\}$

Dynamic Connectivity

It can be used to mode many different problems with different kinds of objects:

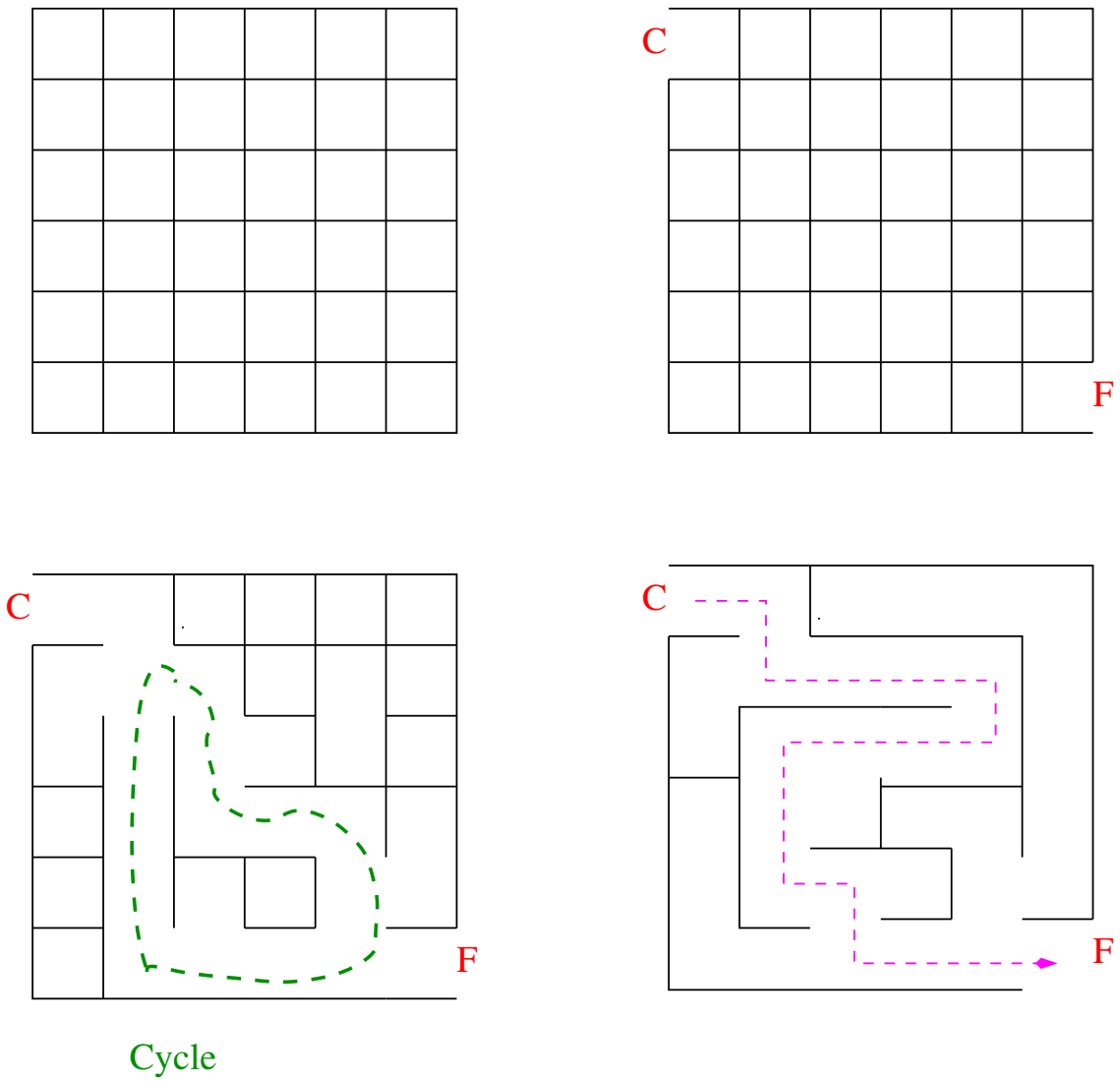
Is there a path C to F?



Building Mazes

- Start from a grid G with n cells and E wall between the cells
- Build a maze by erasing walls
- Define an start C and an end F
- Iterate picking random walls and deleting them, taking care. of not picking a wall separating adjacent vertices, which are in the same component: This is to avoid cycles.
- Do not delete boundary edges (except for C and F).
- Every cell should be reachable from every other cell.
- The paths should define a tree.

Building paths in Mazes



Building Mazes: Union-find

Given a grid $G = (V, E)$, where V are the n cells and the m valls $E = \{(x, y) \mid x, y \in V\}$, together with a C and a F, we want to output a $R \subset E$, such that removing R produces a valid maze.

1. Consider the set V of cells and number them $[n]$
2. Consider the set of walls $E = \{(i, j)\}$ between adjacent cells $i, j \in V$
   - 2.1 For every cell i MAKESET$(i) = \{i\}$
   - 2.2 Iterate until having a path $C \rightsquigarrow F$
   - 2.3 Take a wall $(i, j) \in E$
   - 2.4 If FIND$(i) \neq$FIND(j) Erase the wall make UNION(FIND(i),FIND(j))
   - 2.5 Enditerate

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

Building Mazes: Algorithm

Recall $|V| = n, |E| = m$:

MAZE $(V, E), C, F,\ R = \emptyset$
for each $x \in V$ **do**
  MAKESET (x)
end for
while $|R| < n - 1$ **do**
  Select uniformly at random $(y, z) \in E$
  Remove (y, z) from E
  if $x = \text{FIND}\ (y) \neq w = \text{FIND}\ (y)$ **then**
    UNION (x, w)
    $R = R \cup \{(y, z)\}$
  end if
end while

Example Mazes: First iterations

After MAKESET: $V = \{\{1\}, \{2\}, \ldots, \{36\}\}$
Edge $(7, 13)$: $V = \{\{1\}, \{2\}, \ldots, \{7, 13\}, \ldots \{36\}\}$
Edge $(2, 8)$: $V = \{\{1\}, \{2, 8\}, \ldots, \{7, 13\}, \ldots \{36\}\}$
Edge $(7, 8)$: $V = \{\{1\}, \{2, 7, 8, 13\}, \ldots, \{36\}\}$

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

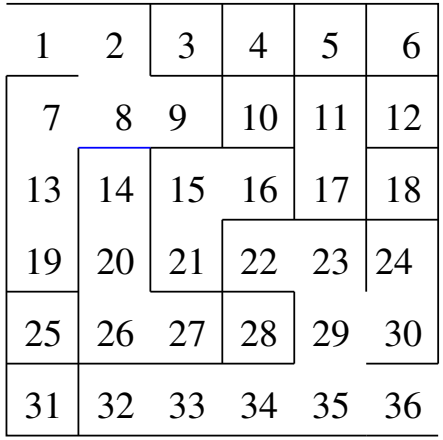
Example Mazes: Intermediate

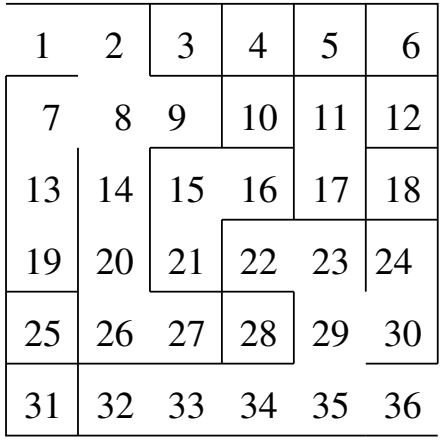
Start

$V = \{\{1, 2, 7, 8, 9, 13, 19\}, \{3\}, \{4\}, \ldots, \{11, 17\}, \{14, 20, 26, 27\}, \{15, 16, 21\}, \{22, 23, 24, 29, 30, 32, 33, 34, 35, 36\}\}$

choose $(8, 14) \Rightarrow$ FIND(8)$=7 \neq 20=$ FIND $(14) \Rightarrow$ UNION$(7, 14)$

New $V = \{\{1, 2, 7, 8, 9, 13, 14, 19, 20, 26, 27\}, \ldots\}$

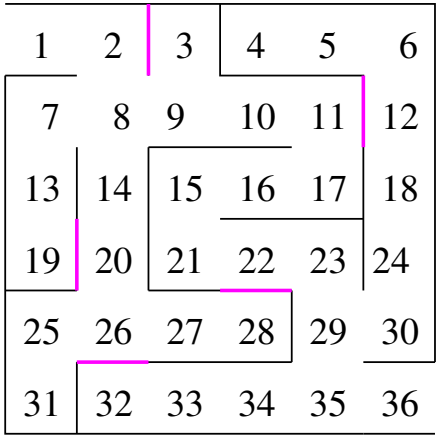




Example Mazes: Intermediate

Final configuration:
$V = \{1, 2, \ldots, 7, 8, \ldots, 36\}$

At the given pattern, the edges $\{(2,3), (11,12), (19,20), (26,32)\}$, should not be selected



How do we implement union find?

Amortized analysis

(See for ex. Sect. 17-1 to 17.3 in CLRS)

- An amortized analysis is any strategy for analyzing a sequence of operations on a Data Structure, to show that the "average" cost per operation is small, even though a single operation within the sequence might be expensive.
- An amortized analysis guarantees the average performance of each operation in the worst case.
- The easier way to think about amortized analysis is to consider the number of steps averaged over all the sequence of operations.
- Amortization gives us a procedure to do an average-case analysis without using any probability.

Dynamic table problem

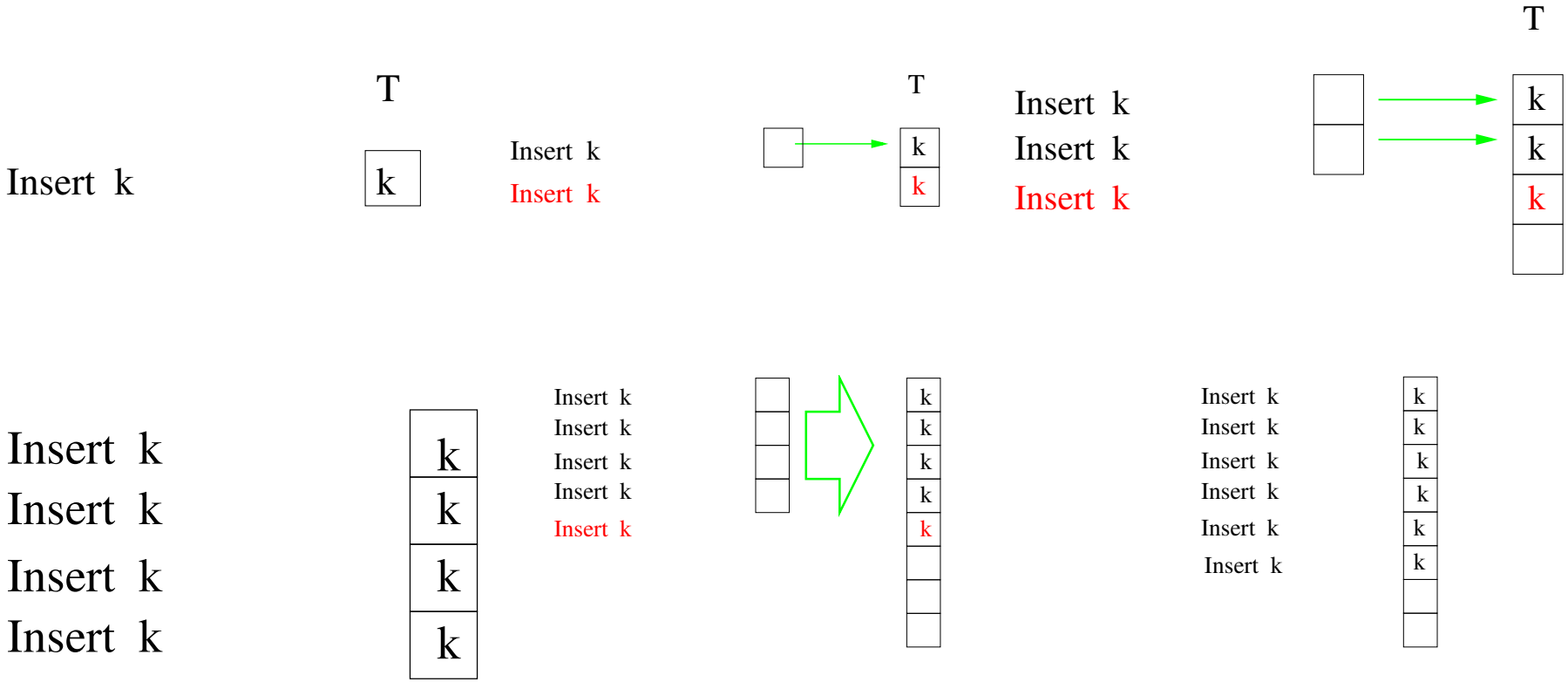
- Tables are implemented as vectors with contiguous memory area to store objects.
- In many applications, it is not known in advance the size of the table, it may happen we run out of space in the middle of an application and dynamically we must increase the size. (Data streaming, Online algorithms)
- *Want to design an efficiency memory allocation strategy for allocating new space in a table* (malloc, new)
- Each time the number of elements in a table is larger than the current size, we have to construct a new table with double size, copy the contains into the new space, delete the old table.
- Problem: We want to store n streaming keys in a table of size m. The goal is to make the m as small as possible, but large enough so that it won't overflow.

Dynamic tables

- We start with table of size 2, and double if at the i-th. operation the table overflows: i.e. $i = 2^x + 1$.
- Whenever the table overflows:
  1. Allocate a new array of double size.
  2. Move all items from the old table into the new one.
  3. Free storage.
- If n operations are performed, the worst case for insertions could have cost $= \Theta(n)$, then the cost of doing n insertions is $n \times \Theta(n) = \Theta(n^2)$.
- WRONG!. That is an over-counting, because we do not expand the table at each of the n insertion operations. So the rough worst case analysis yields $O(n^2)$.

Example

Consider a sequence of insertions from the one-slot, empty table:



Amortized analysis: Aggregate method

For any n, show a sequence of n operations takes worst case time $T(n)$. Then the amortized cost per operation is $T(n)/n$.
Assume we start with table of size 2. Let c_i be the cost of the ith insertion. If $i = 2^x + 1$ then $c(i) = i$, otherwise $c(i) = 1$.

i	1	2	3	4	5	6	7	8	9	10
$\lvert A \rvert$	2	2	4	4	8	8	8	8	16	16
$c(i)$	1	2	3	1	5	1	1	1	9	1

$$T(n) = \sum_{k=0}^{\lfloor \lg n \rfloor} (2^k + 1) + \sum_{i=1 \& i \neq 2^k+1}^{n} 1 \leq 2^{\lfloor \lg n \rfloor + 1} + n = \Theta(n).$$

Thus the amortized cost of each dynamic-table operation is

$$\frac{\Theta(n)}{n} = \Theta(1).$$

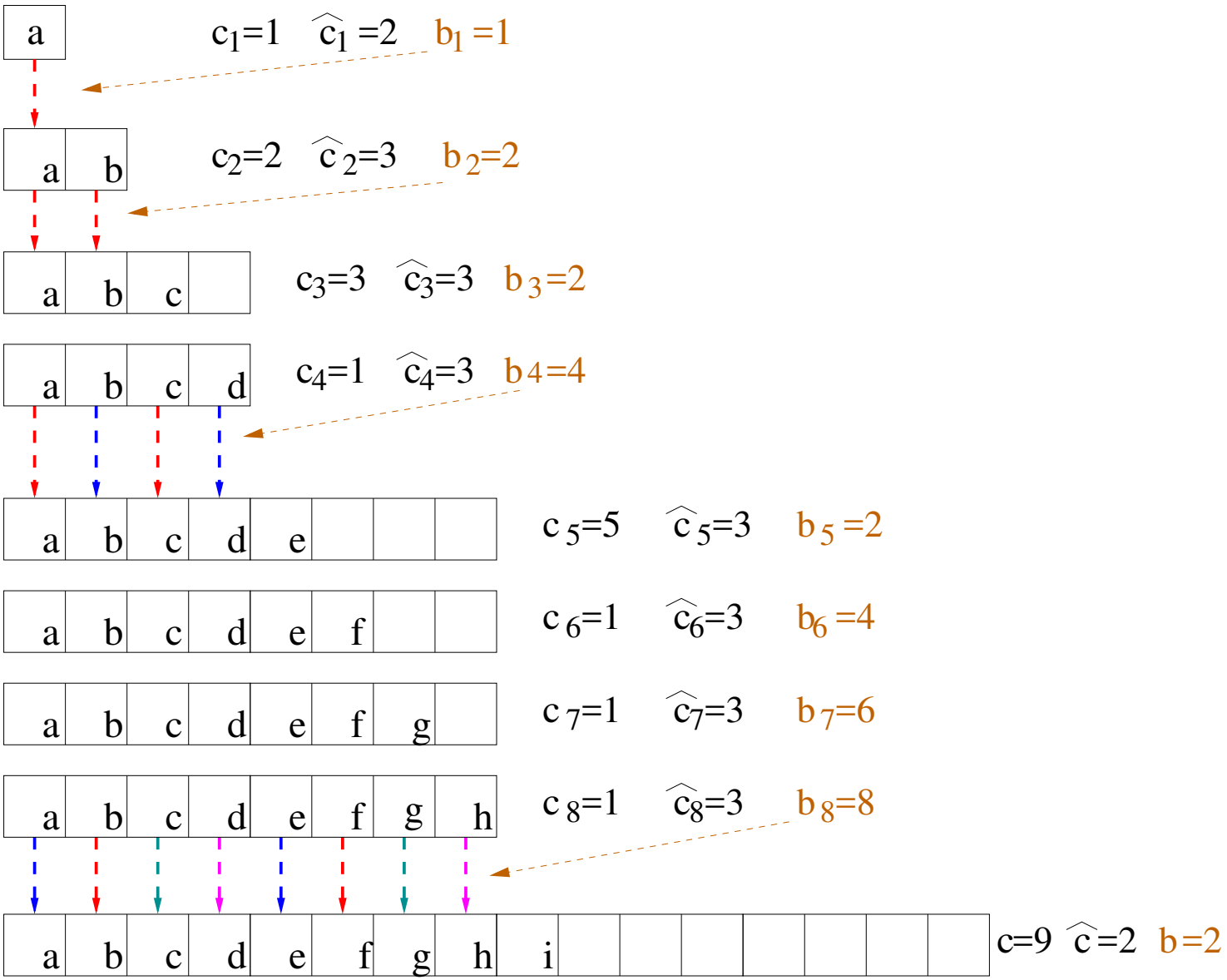
Amortized analysis: Accounting method

- Assign to each different operation i a credit $\hat{c}_i$ ฿.
- Notation In operation i, c_i denotes the real cost and $\hat{c}_i$ denotes the credit and b_i is the resulting balance after implementing the operation (฿).
- The cumulative $\hat{c}_i$ is denote as the amortized cost.
- Notice that for the most expensive operations (copying contents into a larger table, when $i = 2^x + 1$ we must use the credit to amortize the costs of copying from the old table to the new one.
- At each step i, the amortized cost $>$ real cost, the difference b_i, is saved as credit, to be used later.

Dynamic tables: Accounting method

- We want to show that for all sequence of $m = f(n)$ operations, the sequence of $\hat{c}_i$ is an UB to the sequence of c_i,
$$\forall m > 0, \sum_{j=1}^{m} \hat{c}_j \geq \sum_{j=1}^{m} c_j.$$
- The goal is to keep the UB $\hat{c}_i$ as close to the real cost, as possible.
- If $i = 2^x + 1$, c_i is the expense of copying the previous elements and writing the new element, otherwise c_i is the expense of incorporating the new element.
- Charge $\hat{c}_1 = \hat{c}_2 = 2$Ƀ, for each other insertion $i > 2$, $\hat{c}_i = 3$Ƀ.
- At each operation $i > 2$, from $\hat{c}_i = 3$Ƀ, 1Ƀ pays for the insertion and if $i = 2^x + 1$, each register pays 1Ƀfor copying itself and and 1Ƀ to copy one of the earlier registers that have 0 Ƀ.

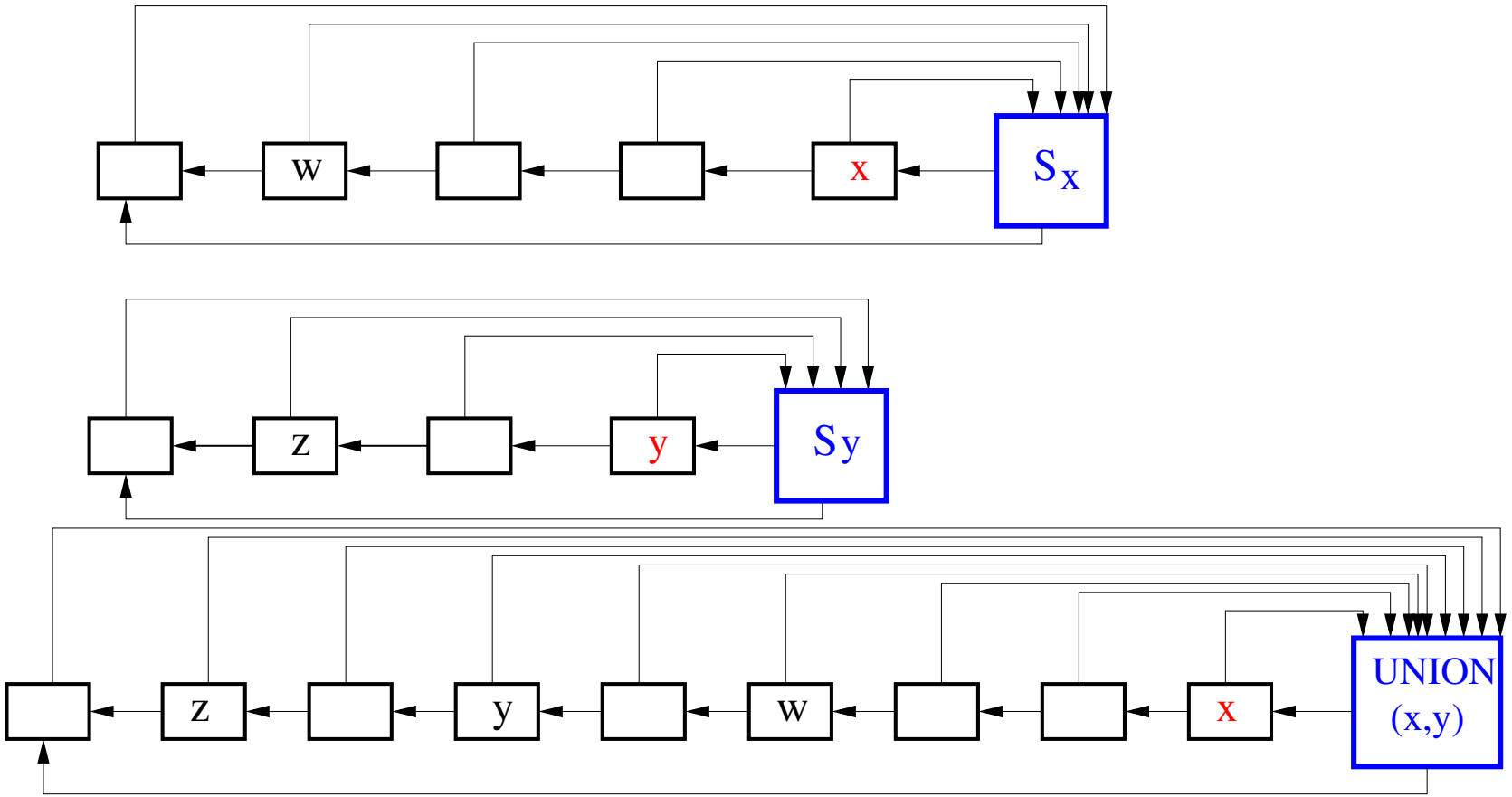
Example


a
$c_1=1$ $\widehat{c}_1=2$ $b_1=1$
a b
$c_2=2$ $\widehat{c}_2=3$ $b_2=2$
a b c
$c_3=3$ $\widehat{c}_3=3$ $b_3=2$
a b c d
$c_4=1$ $\widehat{c}_4=3$ $b_4=4$
a b c d e
$c_5=5$ $\widehat{c}_5=3$ $b_5=2$
a b c d e f
$c_6=1$ $\widehat{c}_6=3$ $b_6=4$
a b c d e f g
$c_7=1$ $\widehat{c}_7=3$ $b_7=6$
a b c d e f g h
$c_8=1$ $\widehat{c}_8=3$ $b_8=8$
a b c d e f g h i
$c=9$ $\widehat{c}=2$ $b=2$

Union-Find implementation: Linked list

Given $\{S_1, \ldots, S_k\}$,

- Each set S_i represented by a linked list, with a header
- The *representative* of S_i is defined to be the element at the head of the list representing the set.



Union-find: Linked-list

For the implementation and complexity of union-find, see for ex. Sec. 5.1.4 of Dasgupta, Papadimitriou, Vazirani.

Given sets S_x and S_y, we use the following 3 operations:

- MAKESET (x): Initializes x as a lone list. Worst time $\Theta(1)$.
- UNION (z, w): Find the representative y, and point to the tail of the list S_x implementing $S_x \cup S_y$.
- FIND (z): Goes from z to the head of S_x and then to the representative of the set. Worst case $O(1)$.

Complexity of UF by linked-lists: Worst case analysis

We have a set of n elements $\{x_1, \ldots, x_n\}$ and m applications of MAKESET and UNION. Notice $m = 2n - 1$:

1. We start with $x_1, x_2, \ldots, x_n$, do n MAKESET. Total cost $O(n)$,
2. do UNION(x_1, x_2), UNION(x_2, x_3), $\ldots$, UNION(x_{n-1}, x_n),

In worst case the cost is $n + \sum_{i=1}^{n-1} i = \Theta(n^2)$.

Amortized analysis: A quick view

Using the aggregate amortized analysis we get that the average cost of each operation is $\Theta(n^2)/(2n-1) \sim \Theta(n)$.

Union-find: Linked-list

We use the following heuristic to implement UNION (z, w):
Append the smallest list to the larger one.
This implies that all pointers in the shortest set must be updated.

- MAKESET (x): Initializes x as a lone list. Worst time $\Theta(1)$.
- UNION (z, w): Find the representative y, and point to the tail of the list S_x implementing $S_x \cup S_y$. Worst time $O(\min\{|S_x| + |S_y\}|)$. As it could be $|S_x| = |S_y| = n/2$, then UNION needs $O(n)$ steps.
- FIND (z): Goes from z to the head of S_x and then to the representative of the set. Worst case $O(1)$.

Complexity of UF by linked-lists: Amortized analysis

Theorem *Using the linked-list implementation of UF, with the max length heuristic for* UNION, *a sequence of* m MAKESET, UNION, FIND *operations,* n *of which are* MAKESET, *takes* $O(n + m \lg n)$ *steps.*
Moreover, the amortized running time of each UNION *operation is* $O(\log n)$, *and the amortized running time for each* MAKESET *and* FIND *is* $\Theta(1)$.

Complexity of UF by linked-lists: Proof of the Theorem

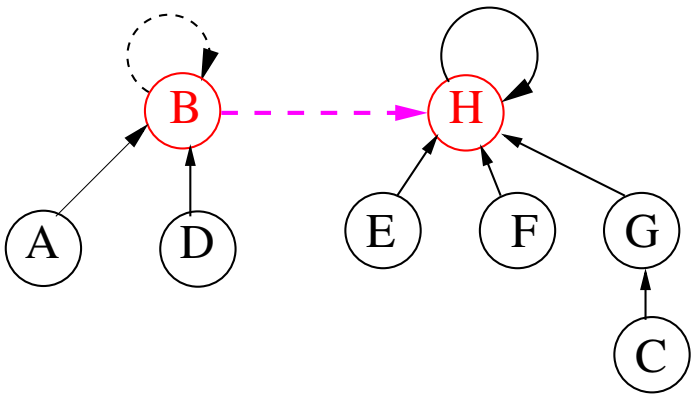
Proof

- If we have n elements, the number of *UNION* operations is $\leq n - 1$.
- Each time we do UNION(x, y), we double the size of the smaller set,
- therefore any element x will update its pointers a maximum of $\lg n$ times,
- therefore the total cost of updating pointers due to the application of *UNION* is $O(n \lg n)$.
- Therefore, the total time for the entire m-sequence is $O(n + m \lg n)$.
- As number of UNIONS is $\leq n - 1$, The amortized cost for each UNION operation is $O(n \lg n / n) = O(\lg n)$.
- The amortized cost for each MAKESET and FIND operation is $\Theta(1)$. $\square$

Union-Find implementation: Link by size of forest of Trees

- Stating from initial n singleton trees, represent each set as a tree of elements, where the root contain the representative of the tree.
- As a DS, this representation of a tree has only parent links, and does not provide a way to access the children of a given node.

• MAKESET (x): $\Theta(1)$
• FIND (z): find the root of the tree containing z. $\Theta(\text{height})$
• UNION (x, y): make the root of the tree with less elements point to the root of the tree with more elements.



Complexity of Link by size

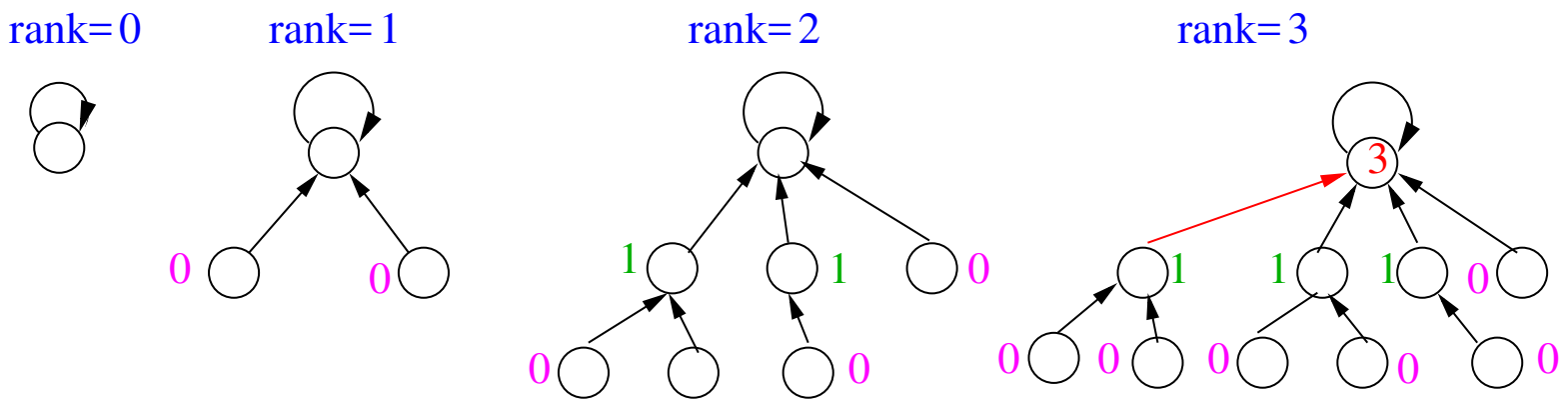
- The cost of MAKESET(x) is $\Theta(1)$ and the cost of FIND(z) is $O(\lg n)$
- Notice that for x, y representatives of S_x and S_y, the cost of UNION (x, y) is $\Theta(1)$.
  If z, w are not representatives, to implement UNION (w, z) we must first do FIND(z) and FIND(w).

Doing a sequence of m MAKESET, FIND, UNION operations, starting from n elements, using the link-by-size tree implementation of UF structure, takes $O(n + m \lg n)$ steps.

So the complexity of UF is the same for both implementations: Link by size of forest of Trees and linked-list with the proposed heuristic.

First Heuristic: Link by rank

- Define the rank $r(x)$ as height of subtree rooted at x.
- Notice It is possible for two trees that the one with less nodes has higher root than the one with less nodes.
- Any singleton element has rank=0.
- Inductively as we join trees, only the rank of the root increases.

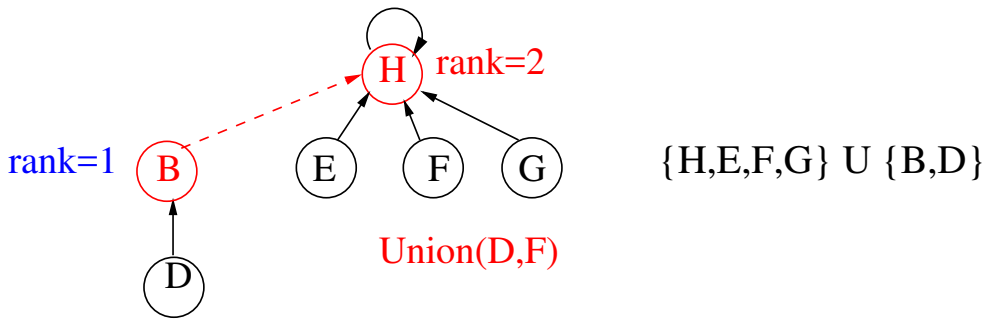


Trees: Link by rank

Union rule: Link the root of smaller rank tree to the root of a tree with larger rank.

In case the roots of both trees have the same rank, choose arbitrarily and increase $+1$ the rank of the winer.

Except for the root, a node does not change rank during the process



• UNION (x, y): is $\Theta(1)$ if x, y are roots, otherwise we have to apply first climbs to the roots of the tree containing FIND(x) and FIND(y), which takes Θ(height) steps.

Link by rank

Maintain an integer rank for each node, initially 0. Link root of smaller rank to root of larger rank; if a tie, increase rank of new root by 1.

Let $p(z)$ be the parent of (z) in the forest, and let $r(z)$ be the rank of (z).

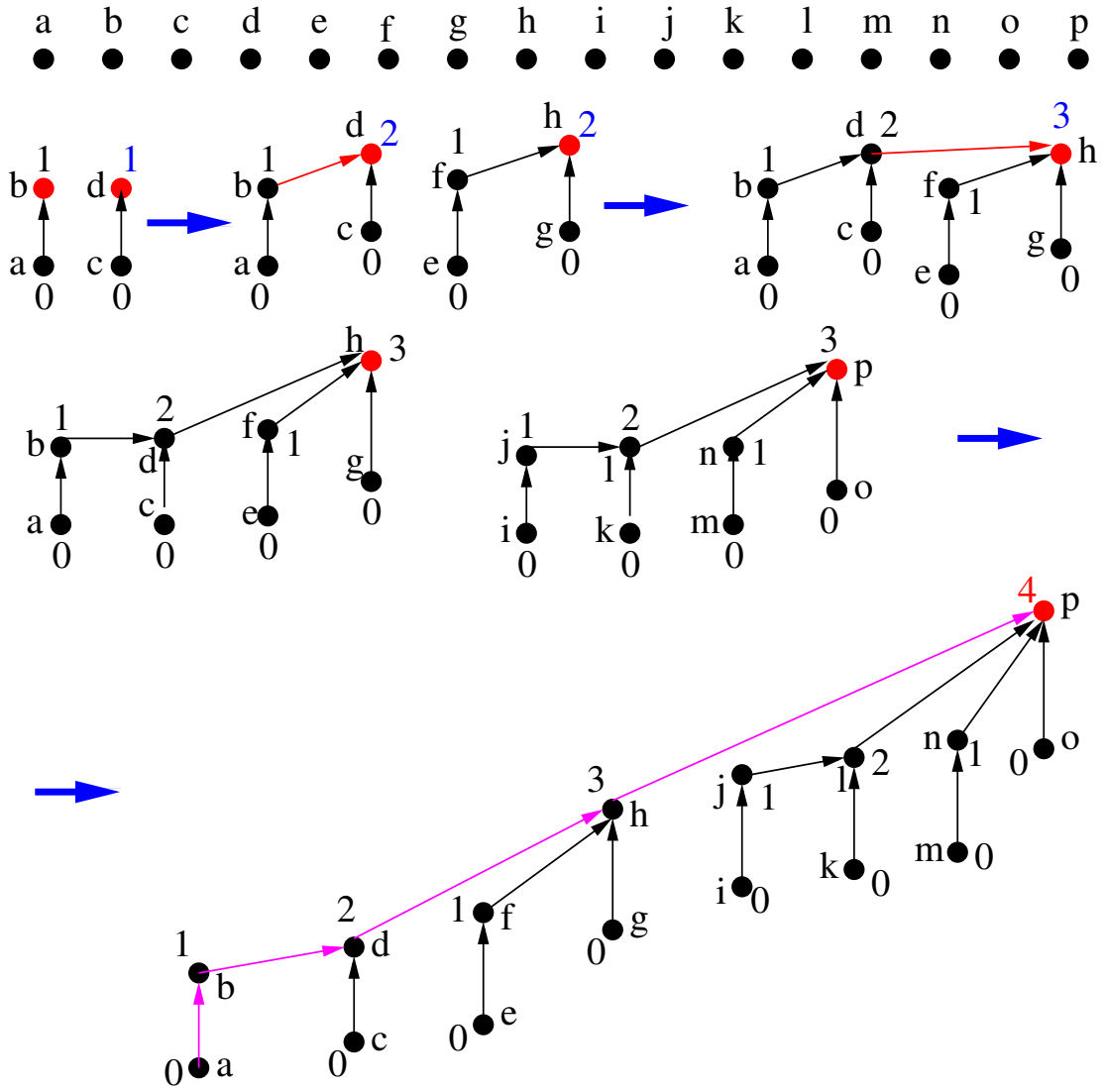
MAKESET (x)
$p(x) = x$
$r(x) = 0$

FIND (z)
while $z \neq p(z)$ **do**
  $z = p(z)$
end while

UNION (x, y)
if $r_x = r_y$ **then**
  STOP
else if $r(r_x) > r(r_y)$ **then**
  $p(r_y) = r_x$
else if $r(r_x) < r(r_y)$ **then**
  $p(r_x) = r_y$
else
  $p(r_x) = r_y$
  $r(r_y) = r(r_y) + 1$
end if

Worst case example

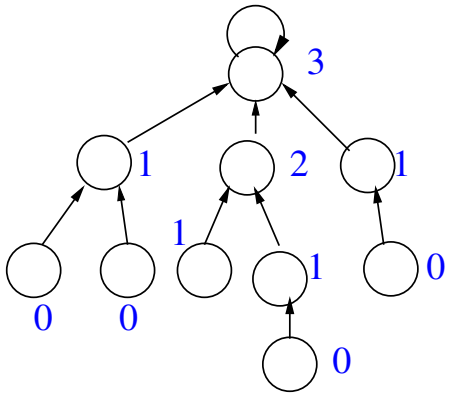
Worst case example that we can have $r(\text{root}) = \lg n$.



Properties of forests constructed by using Link by rank

P1.- If x is not a root then $r(x) < r(p(x))$

P2.- If $p(x)$ changes then $r(p(x))$ increases



P3.- For any node x, let $N(x)$ be the number of nodes in the subtree rooted at x. If $r(x) = k$, then $N(x) \geq 2^k$.

Proof (Induction on k) True for $k = 0$, if true for k then a node of rank k results from the merging of 2 nodes with rank $k-1$, so $2^{k-1} + 2^{k-1} = 2^k$. $\square$

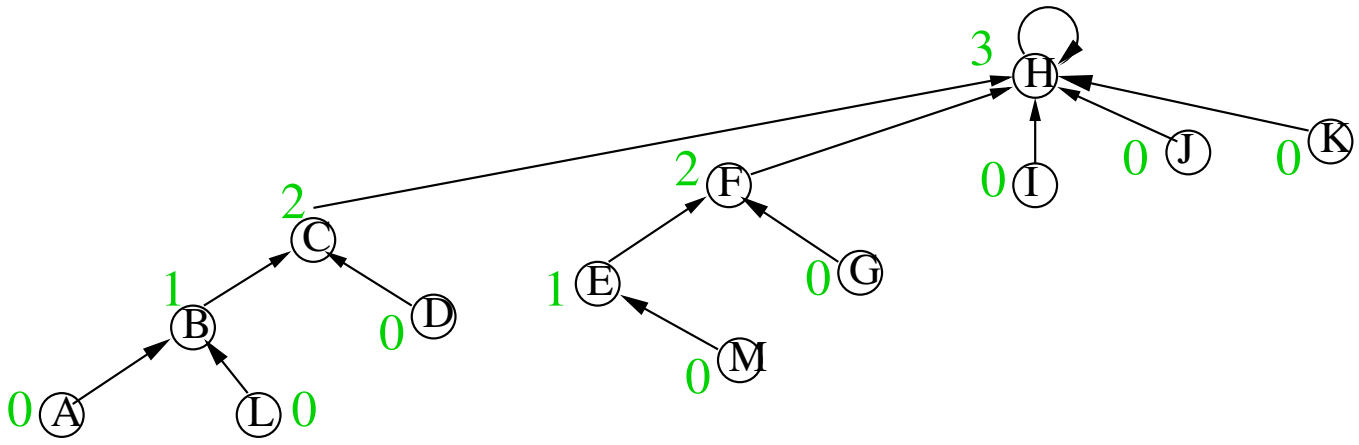
Properties of Link by rank

P4.- The highest rank of a root is $\leq \lfloor \lg n \rfloor$

Proof Follows P1 and P3. $\square$

P5.- For any $r \geq 0$, there are $\leq n/(2^r)$ nodes with rank r.

Proof By (P4) a root x with $r(x) = k$ has $\geq 2^k$ descendants.
Any non-root node y with $r(y) = k'$ has $\geq 2^{k'}$ descendants.
As it is a tree, different nodes with rank $= k$ can't have common descendants. $\square$



rank 0=7; rank 1=2; rank 2=2; rank 3=1

$n/2^k$ rank 0<13; rank1 <6; rank 2<3 rank 3=1

Complexity of Link by rank

As for FIND, the number of steps is bounded by the height of the tree, then

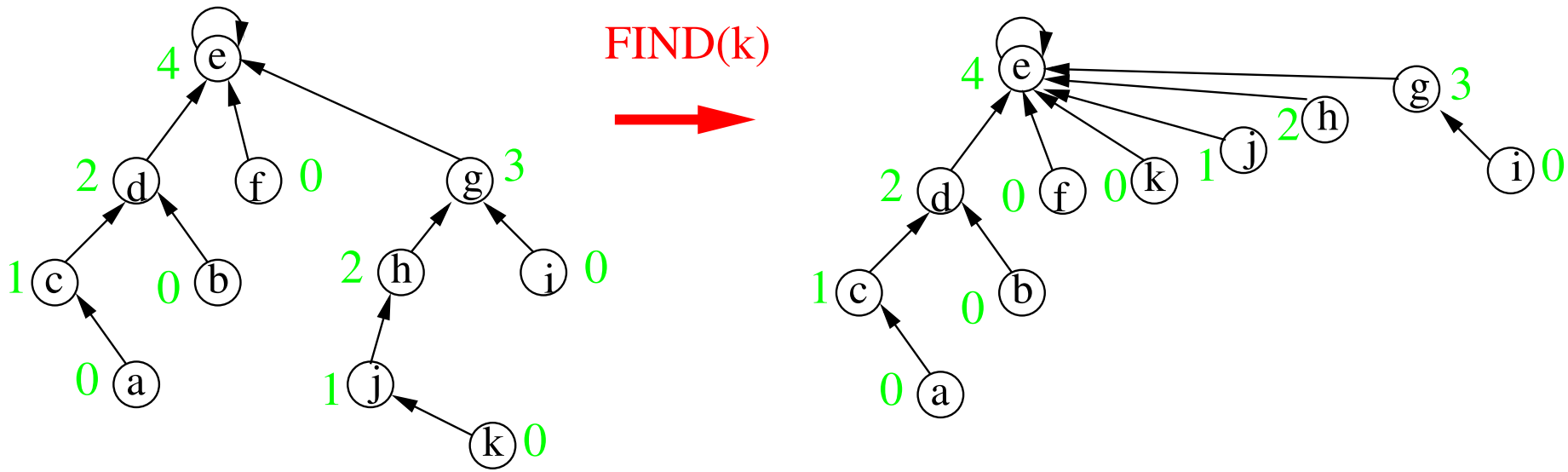
Lemma *Using link-by-rank, each application of* FIND *takes* $O(\lg n)$ *steps.*

Lemma *Starting from an empty data structure with* n *elements, the performance of union-find implemented using link-by-rank, for any* n MAKESET *and any intermixed sequence of* m FIND *and* UNION *operations is* $O(n + m \lg n)$ *steps.*

Final Heuristic: Path compression with link-by-rank

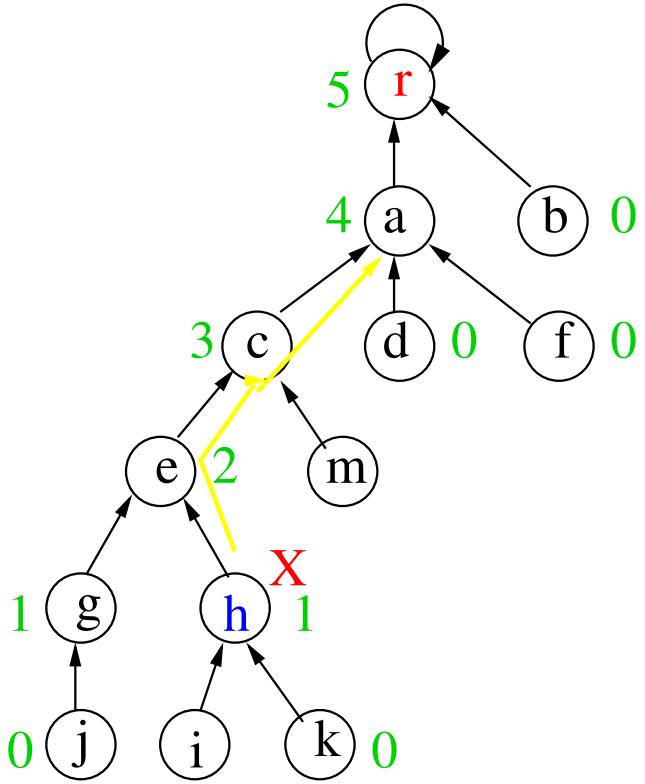
To improve the $O(\log n)$ amortized cost per FIND in union-bound with link-by-rank, we keep the trees "as flat" as possible.

We use the path compression: At each use of FIND(x) we follow all the path $\{y_i\}$ of nodes from x to the root r change the pointers of all the $\{y_i\}$ to point to r.



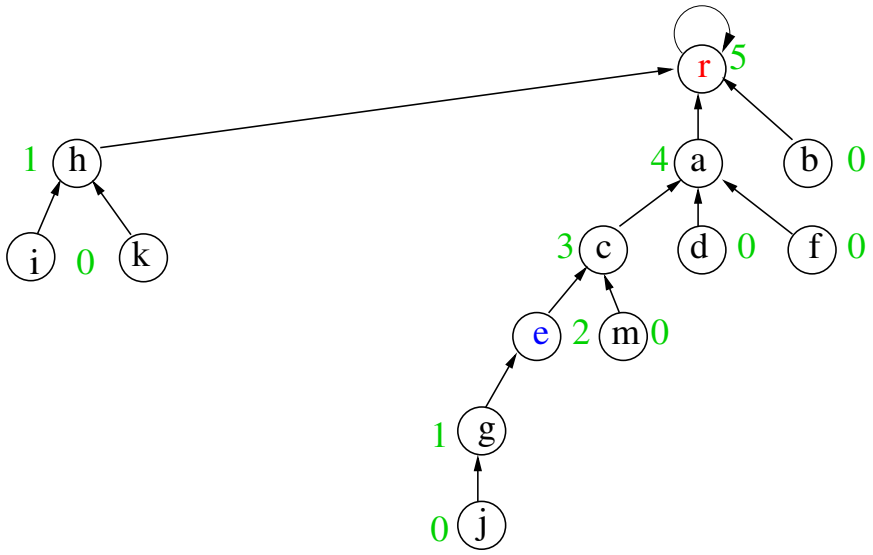
Path compression: Function

FIND (x)
if $x \neq p(x)$ **then**
  $p(x) =$ **FIND** $p(x)$
  return $p(x)$
end if



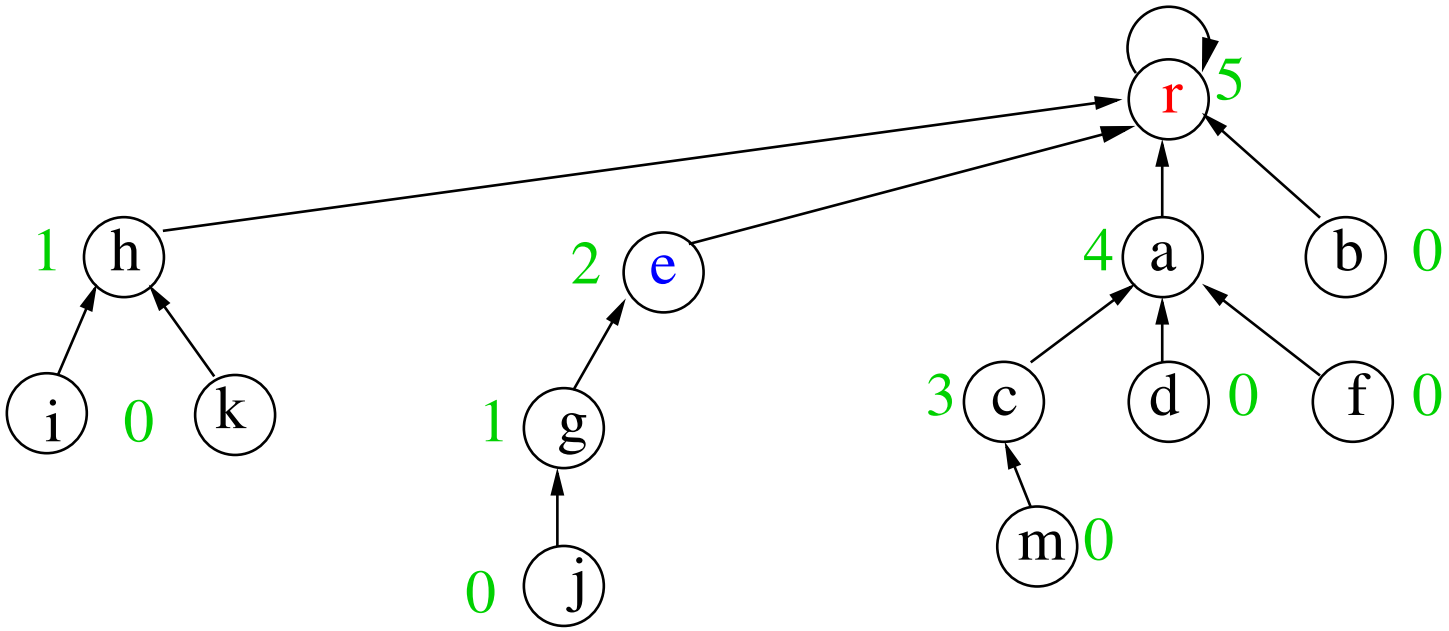
Path compression: Function

FIND (x)
if $x \neq p(x)$ **then**
  $p(x) =$ **FIND** $(p(x))$
  return $p(x)$
end if



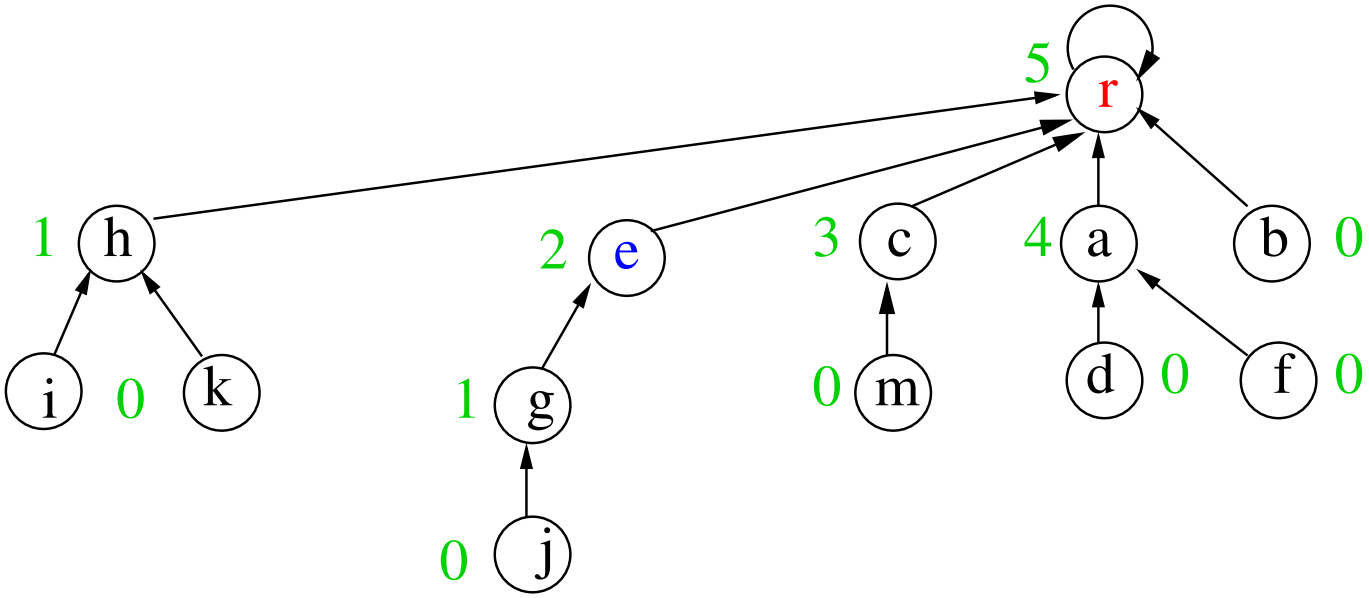
Path compression: Function

FIND (x)
if $x \neq p(x)$ **then**
  $p(x) =$ **FIND** $(p(x))$
  return $p(x)$
end if



Path compression: Function

FIND (x)
if $x \neq p(x)$ **then**
  $p(x) =$ **FIND** $(p(x))$
  return $p(x)$
end if



Union-Find: Link by rank with path compression

This implementation of the data structure is the one that reduces the complexity of making a sequence of m UNION and FIND operations.

Key Observation: Path compression does not create new roots, change ranks, or move elements from one tree to the another

As a corollary, the properties of Link by rank also hold for this implementation.

Notice, the FIND operations only affect the inside nodes, while the UNION operations only affect roots.

Thus, compression has no effect on UNION operations.

The iterated logarithm

The iterated logarithm $\lg^*(n)$ is defined: $\lg^*(n) = k$ if k is the smallest integer such that $\lg^k n = \Theta(1)$.

Recall: $\lg^k n = \underbrace{\lg(\lg(\lg(\cdots(\lg n)\cdots)))}_{k-\text{times}}$.

The formal definition:

$$\lg^*(n) = \begin{cases} 0 & \text{if } n \leq 1, \\ 1 & \text{if } n = 2, \\ 1 + \lg^*(\lg(n)) & \text{if } n > 2. \end{cases}$$

n	$\lg^* n$
1	0
2	1
[3,4]	2
[5,16]	3
[17,65536]	4
[65537,10^{19728}]	5

Why for all practical purposes $\lg^*(n) \leq 5$

We have that if $\lg^*(n) \leq 5$ then for all $n \leq 2^{2^{16}} \sim 10^{19728}$.

Intuitive argument: If we consider that in any computer, each memory bit has size ≥ 1 atoms, using the canonical estimation that the number of atoms in the universe is $\leq 10^{83}$, we can conclude the size of any computer memory is $< 10^{83}$. Therefore it is impossible with today technology, that you can manipulate sets with size 10^{83}

Therefore, we can consider that for any n $\lg^*(n) \leq 5$.

Main result

The following result was due to J. Hopcroft and J. Ullman: SIAMJC 1973

Theorem

Starting from an empty data structure with n disjoint single sets, link-by-rank with path compression performs any intermixed sequence of m FIND *and* UNION *operations in $O(m \lg^* n)$ steps.*

The proof uses an aggregate amortized analysis argument: look at the sequence of FIND and UNION operations from an empty and determine the average time per operation. The amortized costs turns to be $\lg^* n$ (basically constant) instead of $\lg n$.

Recall: Properties 1-6

1. P-1 If x is not a root then $\text{rank}(x) < \text{rank}(\text{parent}(x))$
2. P-2 If $r(x)$ changes then it is a root and $r(x)$ increases.
3. P-3 A root with rank k has $N \geq 2^k$ nodes in its tree.
4. P-4 If there are n elements, their rank values are $\leq \lfloor \lg n \rfloor$.
5. P-5 For any integer $k \geq 0$ there are $\leq n/2^k$ nodes with rank k.
6. P-6 New The ranks of all nodes are unchanged by path compression, those values can not longer be interpreted as tree heights.

Analysis

- Divide non-zero ranks into the following intervals:

$$\underbrace{\{1\}}_{k=0}, \underbrace{\{2\}}_{k=1}, \underbrace{\{3,4\}}_{k=2}, \underbrace{\{5,\ldots,16\}}_{k=3}, \underbrace{\{65537,\ldots,2^{65536}\}}_{k=4}, \ldots \underbrace{\{k+1,\ldots,2^k\}}_{k\text{group}}.$$

- By P-4, every non-zero rank in a Union-find implemented with link by rank with path compression falls within the first $\lg^* n$ intervals.
- We give a credit of 2^k Ƀ to any non-root node with rank within $\{k+1,\ldots,2^k\}$
- By P-6, the number of nodes with rank $\geq k+1$ is $\leq \frac{n}{2^{k+1}} + \frac{n}{2^{k+2}} + \cdots \leq \frac{n}{2^k}$, then nodes in interval k need $\leq n$ Ƀ.
- As there are $\leq \lg^* n$ intervals:
  The number of Ƀ given to all nodes is $\leq n \lg^* n$.

Summary

Given from empty data structure with n disjoint single sets, on which there is a sequence of m FIND and UNION operations, the following table gives summary of the time-complexity for the different implementations of union-bound:

Implementation	cost
Linked list	$O(n + m \lg n)$
Link by size	$O(n + m \lg n)$
Link by rank	$O(n + m \lg n)$
Rank + path compression	$O(n + m \lg^* n) \simeq O(n + m)$

Notice UF in inherently sequential method, so parallel would not help that much.

Back to Kruskal: MST induces am equivalence classes partition of V of G

Notice that Kruskal evolves by building clumps of trees and merging the clumps into larger clumps, taking care there are not cycles.

In trees, the connectivity relation is an equivalence relation, two nodes are connected if there is only one path between them.

Given an undirected $G = (V, E)$, the MST T on $G = (V, E)$, induces an equivalence relation between the vertices of V: $u\mathcal{R}v$ iff there a T-path between u and v:

$\mathcal{R}$ partition the elements of V in equivalence classes, which are connected components without cycles

Union-Find implementation for Kruskal

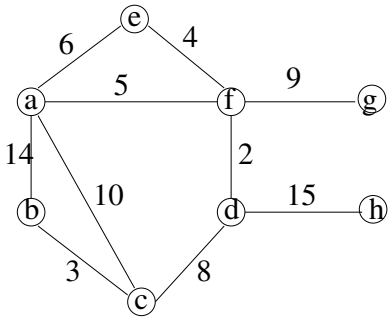
```
MST (G(V, E), w, r), |V| = n, |E| = m
Sort E by increasing weight: {e_1, ..., e_m}
T := ∅
for all v ∈ V do
    MAKESET(v)
end for
for i = 1 to m do
    Chose e_i = (u, v) in order from E
    if FIND(x) ≠ Find(y) then
        T := T ∪ {e_i}
        UNION(u, v)
    end if
end for
```

If we use the link-by-rank with path compression implementation, the disjoint set operations take $O(m \lg^* n) = O(m)$.

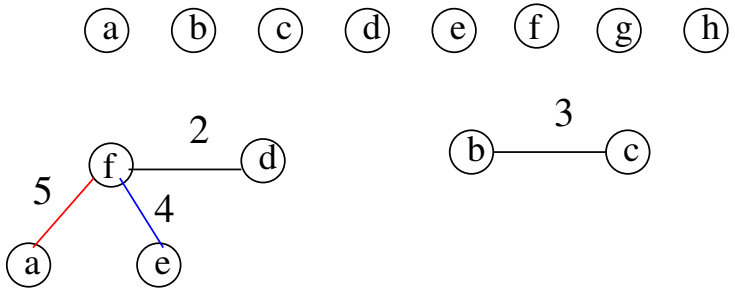
But due to the sorting instruction, the overall complexity of Kuskal still is $O(m \lg n)$.

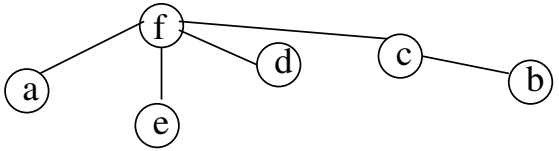
Unless we use a range of weight that allow us to use RADIX.

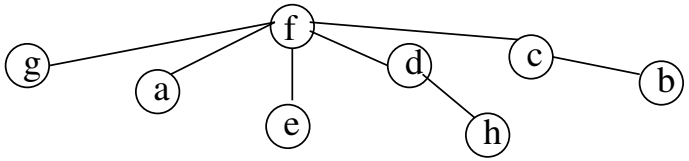
Example of Kruskal using Union-Find



$$E = \{(f,d),(c,b),(e,f),(a,f),(a,e),(c,d),(f,g),(a,c),(a,b),(d,h)\}$$







Greedy and approximation algorithms

- Many times the Greedy strategy yields a feasible solution with value which is near to the optimum solution.
- In many practical cases, when finding the global optimum is hard, the greedy may yield a *good enough* feasible solution: An approximation to the optimal solution.
- An approximation algorithm for the problem always computes a close valid output. Heuristics also could yield approximated solutions, but they are not algorithms.
- Greedy is one of the algorithmic techniques that produce approximations algorithms, but it is not the only one.

Greedy and approximation algorithms

- For any optimization problem, let $c(*)$ be the value of the optimization function, let $\mathcal{A}px$ be the approximation algorithm and let OPT be the exact algorithm, which could have exponential cost.
- For any instance a we want to design an approximation algorithm that do produce a solution $c(\mathcal{A}px(e))$, as close as possible to the unknown optimal solution $c(\mathrm{OPT}\,(e))$.
- Giving a problem that it is not in P, therefore we don't know if it has polynomial time algorithms, we want to design algorithms that are fast (polynomial) and that output solutions as close as it is possible to $c(\mathrm{OPT}\,(e))$.

Formal definition

Given an optimization problem, an α-approximation algorithm $\mathcal{A}px$ computes a solution whose cost is within an $\alpha \geq 1$ factor of the cost $\mathrm{OPT}\,(e)$:

$$\frac{1}{\alpha} \leq \frac{c(\mathcal{A}px(e))}{c(\mathrm{OPT}\,(e))} \leq \alpha.$$

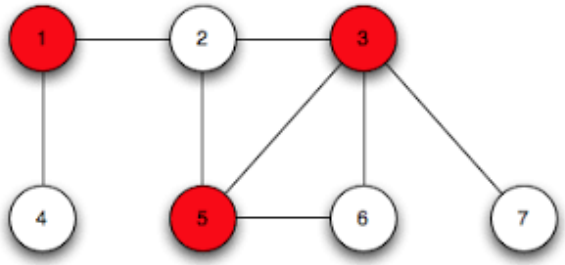
α is called the approximation ratio.

Notice, α measures the factor by which the cost output of $\mathcal{A}px$ exceeds that of $\mathrm{OPT}\,(e)$, on any input.

The first $\leq$ works for maximization and the second $\leq$ works for minimization.

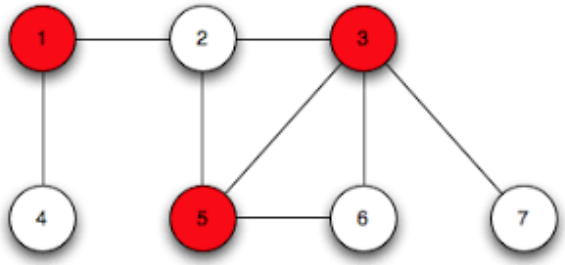
An easy example: Vertex cover

Given a graph $G = (V, E)$ with $|V| = n, |E| = m$ find the minimum set of vertices $S \subseteq V$ such that it covers every edge of G.

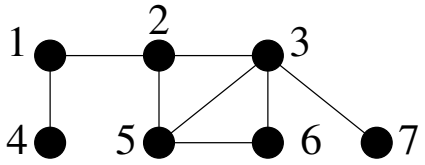


An easy example: Vertex cover

Given a graph $G = (V, E)$ with $|V| = n, |E| = m$ find the minimum set of vertices $S \subseteq V$ such that it covers every edge of G.

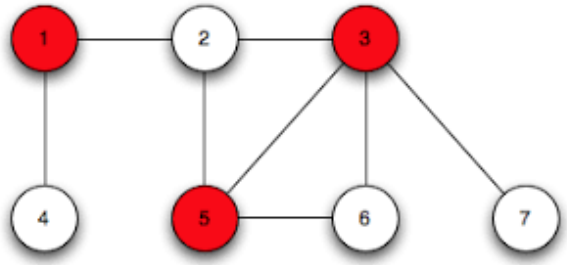


GreedyVC $G = (V, E)$
$E' = E$, $S = \emptyset$,
while $E' \neq \emptyset$ **do**
  Pick $e \in E'$, say $e = (u, v)$
  $S = S \cup \{u, v\}$,
  $E' = E' - \{(u, v) \cup \{\text{edges incident to } u, v\}\}$
end while
return S.

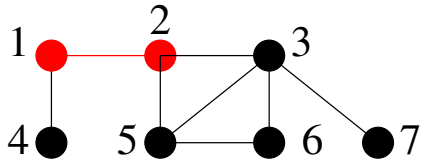


An easy example: Vertex cover

Given a graph $G = (V, E)$ with $|V| = n, |E| = m$ find the minimum set of vertices $S \subseteq V$ such that it covers every edge of G.

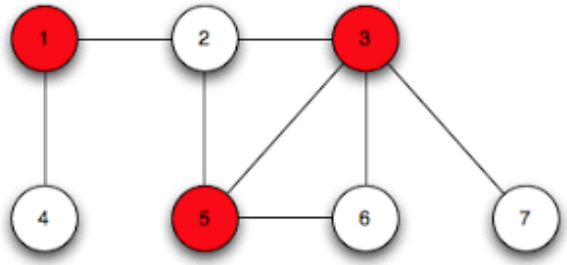


GreedyVC $G = (V, E)$
$E' = E$, $S = \emptyset$,
while $E' \neq \emptyset$ **do**
  Pick $e \in E'$, say $e = (u, v)$
  $S = S \cup \{u, v\}$,
  $E' = E' - \{(u, v) \cup \{\text{edges incident to } u, v\}\}$
end while
return S.

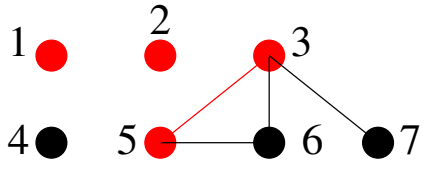


An easy example: Vertex cover

Given a graph $G = (V, E)$ with $|V| = n, |E| = m$ find the minimum set of vertices $S \subseteq V$ such that it covers every edge of G.

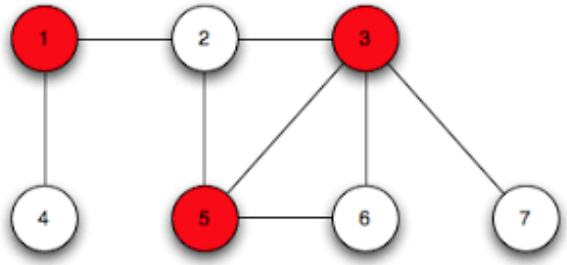


GreedyVC $G = (V, E)$
$E' = E$, $S = \emptyset$,
while $E' \neq \emptyset$ **do**
  Pick $e \in E'$, say $e = (u, v)$
  $S = S \cup \{u, v\}$,
  $E' = E' - \{(u, v) \cup \{\text{edges incident to } u, v\}\}$
end while
return S.

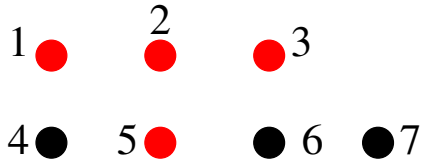


An easy example: Vertex cover

Given a graph $G = (V, E)$ with $|V| = n, |E| = m$ find the minimum set of vertices $S \subseteq V$ such that it covers every edge of G



GreedyVC $G = (V, E)$
$E' = E$, $S = \emptyset$,
while $E' \neq \emptyset$ **do**
  Pick $e \in E'$, say $e = (u, v)$
  $S = S \cup \{u, v\}$,
  $E' = E' - \{(u, v) \cup \{\text{edges incident to } u, v\}\}$
end while
return S.



An easy example: Vertex cover

Theorem The algorithm $\mathcal{A}px$ runs in $O(m+n)$ steps. Moreover, $|\mathcal{A}px(e)| \leq 2|\mathrm{OPT}\,(e)\,|$.

Proof: We use induction to prove $|\mathcal{A}px(e)| \leq 2|\mathrm{OPT}\,(e)\,|$. Notice for every $\{u,v\}$ we add to $\mathcal{A}px(e)$, either u or v are in $\mathrm{OPT}\,(e)$.

Base: If $V = \emptyset$ then $|\mathcal{A}px(e)| = |\mathrm{OPT}\,(e)\,| = 0$.

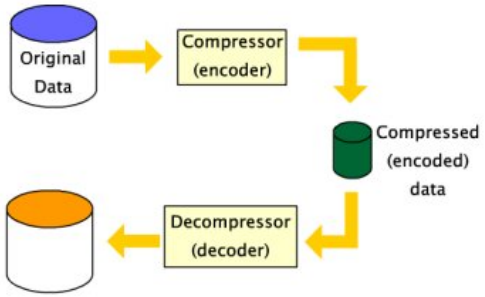
Hipothesis: $|\mathcal{A}px(e - \{u,v\})| \leq 2|\mathrm{OPT}\,(e - \{u,v\})\,|$. Then,

$$\begin{aligned} |\mathcal{A}px(e)| &= |\mathcal{A}px(e - \{u,v\})| + 2 \leq 2|\mathrm{OPT}\,(e - \{u,v\})\,| + 2 \\ &\leq 2(|\mathrm{OPT}\,(e - \{u,v\})\,| + 1) + 2 \leq 2|\mathrm{OPT}\,(e)\,|. \qquad \square \end{aligned}$$

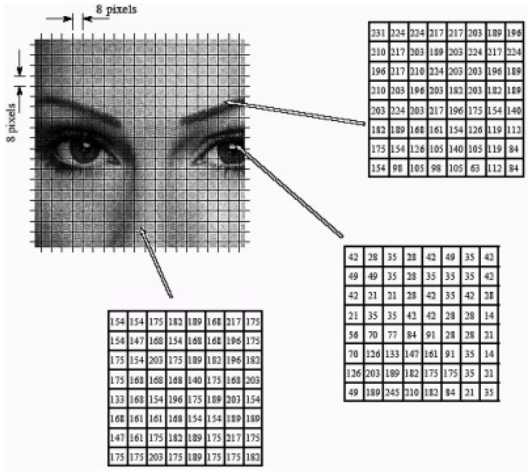
The decision problem for Vertex Cover is NP-complete. Moreover, unless P=NP, vertex cover can't be approximated within a factor $\alpha \leq 1.36$

Data Compression

INPUT: Given a text $\mathcal{T}$ over an finite alphabet Σ
QUESTION: Represent $\mathcal{T}$ with as few bits as possible.

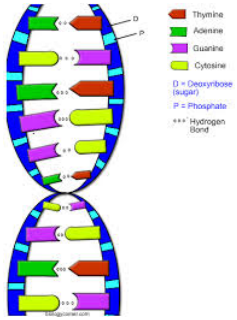


The goal of data compression is to reduce the time to transmit large files, and to reduce the space to store them.
If we are using variable-length encoding we need a system easy to encode and decode.



Example.

$$\underbrace{AAACAGTTGCAT \cdots GGTCCCTAGG}_{130.000.000}$$



- *Fixed-length encoding*: $A = 00$,$C = 01$, $G = 10$ and $T = 11$. Needs 260Mbites to store.
- *Variable-length encoding*: If A appears 7×10^8 times, C appears 3×10^6 times, G 2×10^8 and T 37×10^7, better to assign a shorter string to A and longer to C

Prefix property

Given a set of symbols Σ, a prefix code, is $\phi : \Sigma \to \{0,1\}^+$ (symbols to chain of bits) where for distinct $x, y \in \Sigma$, $\phi(x)$ is not a prefix of $\phi(y)$.

If $\phi(A) = 1$ and $\phi(C) = 101$ then ϕ is no prefix code.

$\phi(A) = 1, \phi(T) = 01, \phi(G) = 000, \phi(C) = 001$ is prefix code.

Prefix codes easy to decode (left-to-right):
00010110011010000010 1

$$\underbrace{000}_{G}\ \underbrace{1}_{A}\ \underbrace{01}_{T}\ \underbrace{1}_{A}\ \underbrace{001}_{C}\ \underbrace{1}_{A}\ \underbrace{01}_{T}\ \underbrace{000}_{G}\ \underbrace{001}_{C}\ \underbrace{01}_{T}$$

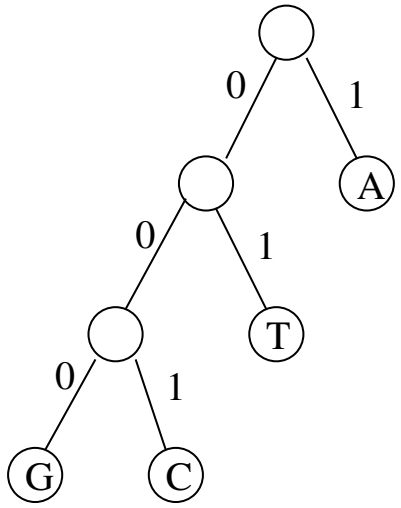
Prefix tree.

Represent encoding with prefix property as a binary tree, the prefix tree:

A prefix tree T is a binary tree with the following properties:

- One leaf for symbol,
- Left edge labeled 0 and right edge labeled 1,
- Labels on the path from the root to a leaf specify the code for that leaf.

For $\Sigma = \{A, T, G, C\}$



Frequency.

To find an efficient code, first given a text S on Σ, with $|S| = n$, first we must find the frequencies of the alphabet symbols.

$\forall x \in \Sigma$, define the frequency

$$f(x) = \frac{\text{number occurrencies of } x \in S}{n}$$

Notice: $\sum_{x \in \Sigma} f(x) = 1$.
Given a prefix code ϕ, which is the total length of the encoding?
The encoding length of S is

$$B(S) = \sum_{x \in \Sigma} n f(x) |\phi(x)| = n \underbrace{\sum_{x \in \Sigma} f(x) |\phi(x)|}_{\alpha} .$$

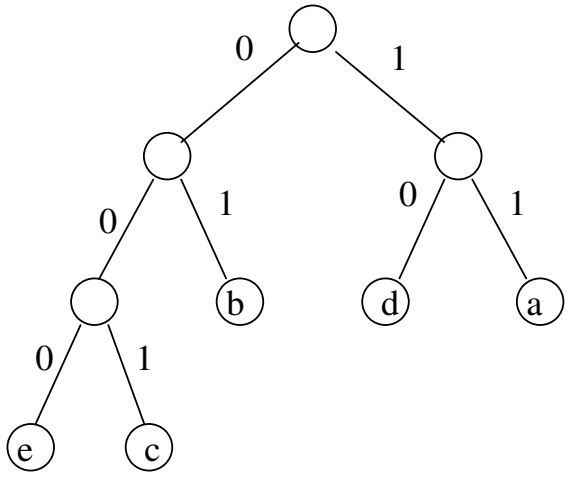
Given ϕ, $\alpha = \sum_{x \in \Sigma} f(x)|\phi(x)|$ is the average number of bits required per symbol.

In terms of prefix tree of ϕ, given x and $f(x)$, the length of the codeword $|\phi(x)|$ is also the depth of x in T, let us denote it by $d_x(T)$.

Let $B(T) = \sum_{x \in \Sigma} f(x) d_x(T)$.

Example.

Let $\Sigma = \{a, b, c, d, e\}$ and let S be a text over Σ.
Let $f(a) = .32, f(b) = .25, f(c) = .20, f(d) = .18, f(e) = .05$
If we use a fixed length code we need $\lceil \lg 5 \rceil = 3$ bits.
Consider the prefix-code ϕ_1:
$\phi_1(a) = 11, \phi_1(b) = 01, \phi_1(c) = 001, \phi_1(d) = 10, \phi_1(e) = 000$

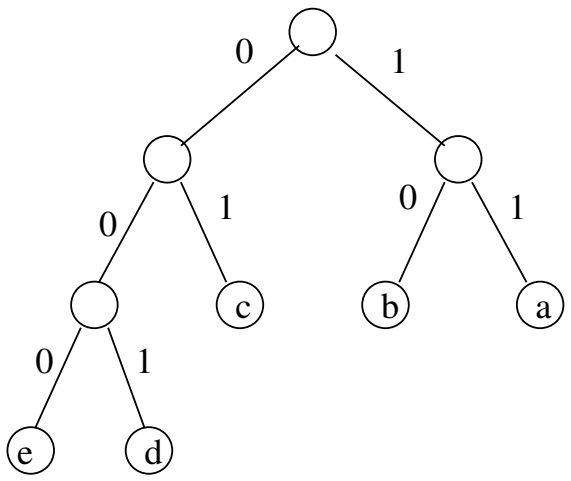


$\alpha = .32 \cdot 2 + .25 \cdot 2 + .20 \cdot 3 + .18 \cdot 2 + .05 \cdot 3 = 2.25$
In average, ϕ_1 reduces the bits per symbol over the fixed-length code from 3 to 2.25, about 25%

Is that the maximum reduction?

Consider the prefix-code ϕ_2:

$\phi_2(a) = 11, \phi_2(b) = 10, \phi_2(c) = 01, \phi_2(d) = 001, \phi_2(e) = 000$



$\alpha = .32 \cdot 2 + .25 \cdot 2 + .20 \cdot 2 + .18 \cdot 3 + .05 \cdot 3 = 2.23$

is that the best? (the maximal compression)

Optimal prefix code.

Given a text, an optimal prefix code is a prefix code that minimizes the total number of bits needed to encode the text.

Note that an optimal encoding minimizes α.

Intuitively, in the T of an optimal prefix code, symbols with high frequencies should have small depth ans symbols with low frequency should have large depth.

The search for an optimal prefix code is the search for a T, which minimizes the α.

Characterization of optimal prefix trees.

A binary tree T is full if every interior node has two sons.

Lemma

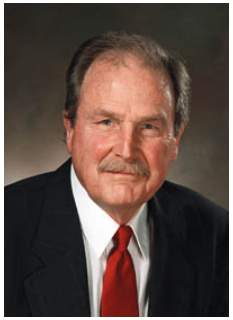
The binary prefix tree corresponding to an optimal prefix code is full.

Proof.

Let T be the prefix tree of an optimal code, and suppose it contains a u with a son v.
If u is the root, construct T' by deleting u and using v com root.
T' will yield a code with less bits to code the symbols.
Contradiction to optimality of T.
If u is not the root, let w be the father of u. Construct T' by deleting u and connecting directly v to w. Again this decreases the number of bits, contradiction to optimality of T. $\square$

Greedy approach: Huffman code

Greedy approach due to David Huffman (1925-99) in 1952, while he was a PhD student at MIT



Wish to produce a labeled binary full tree, in which the leaves are as close to the root as possible. Moreover symbols with low frequency will be placed deeper than the symbol with high frequency.

Greedy approach: Huffman code

- Given S assume we computed $f(x)$ for every $x \in \Sigma$
- Sort the symbols by increasing f. Keep the dynamic sorted list in a priority queue Q.
- Construct a tree in bottom-up fashion, take two first elements of Q join them by a new *virtual node* with f the sum of the f's of its sons, and place the new node in Q.
- When Q is empty, the resulting tree will be prefix tree of an optimal prefix code.

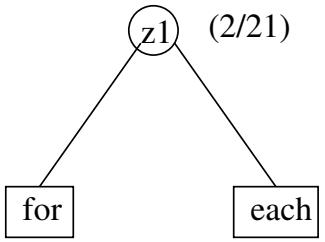
Huffman Coding: Construction of the tree.

Huffman Σ, S
Given Σ and S {compute the frequencies $\{f\}$}
Construct priority queue Q of Σ, ordered by increasing f
while $Q \neq \emptyset$ **do**
  create a new node z
  $x =$Extract-Min (Q)
  $y =$Extract-Min (Q)
  make x, y the sons of z
  $f(z) = f(x) + f(y)$
  Insert (Q, z)
end while

If Q is implemented by a Heap, the algorithm has a complexity $O(n \lg n)$.

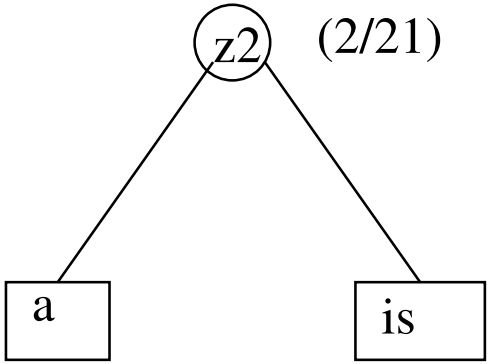
Example

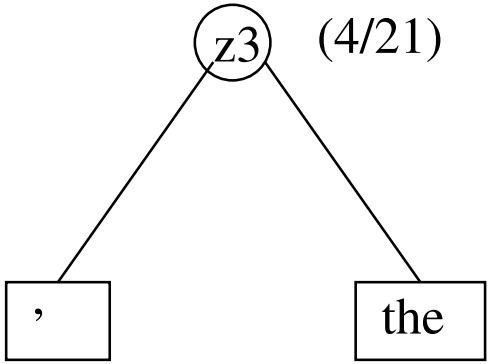
Consider the text: *for each rose, a rose is a rose, the rose*
with $\Sigma = \{$for/ each/ rose/ a/ is/ the/ ,/ $\flat$ $\}$
Frequencies: $f(\text{for}) = 1/21$, $f(\text{rose}) = 4/21$, $f(\text{is}) = 1/21$, $f(\text{a}) = 2/21$, $f(\text{each}) = 1/21$, $f(,) = 2/21$, $f(\text{the}) = 1/21$, $f(\flat) = 9/21$.
Priority Queue:
Q=(for(1/21), each(1/21), a(1/21), is(1/21), ,(2/21), the(2/21), rose(4/21), $\flat$ (9/21))

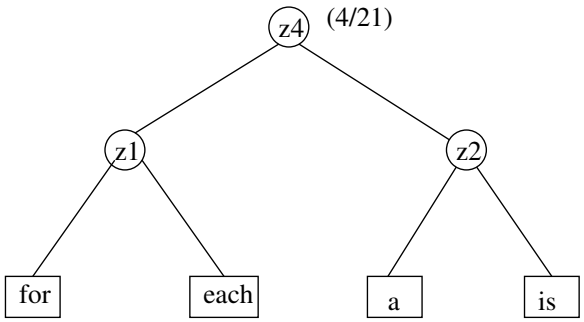


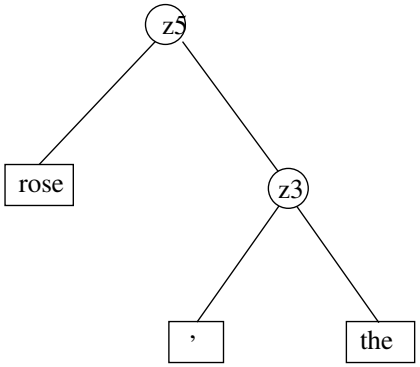
Q=(a(1/21), is(1/21), ,(2/21), the(2/21), z1(2/21), rose(4,21), $\flat$ (9/21))

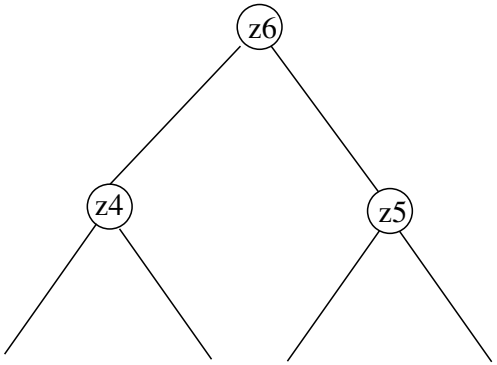
Example.

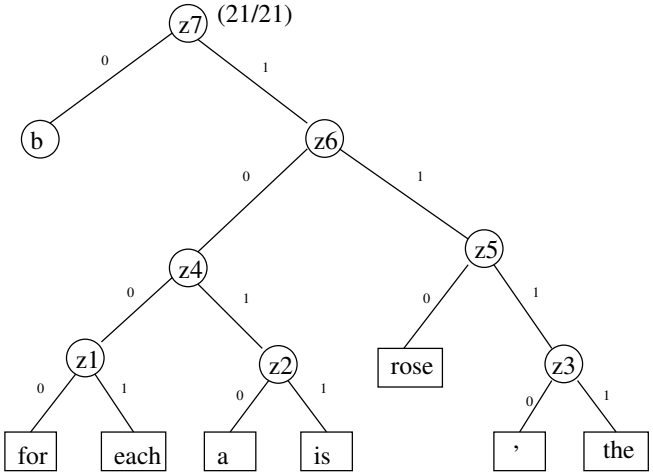

z2
(2/21)
a
is


z3
(4/21)
,
the


z4
(4/21)
z1
z2
for
each
a
is


z5
rose
z3
,
the


z6
z4
z5


z7
(21/21)
0
1
b
z6
0
1
z4
z5
0
1
0
1
z1
z2
rose
z3
0
1
0
1
0
1
for
each
a
is
,
the

Example

Therefore *for each rose, a rose is a rose, the rose* is Huffman codified:
1000010010110111001010011001011010100110111001111 0110

Notice with a fix code we will use 4 bits per symbol $\Rightarrow$ 84 bits instead of the 53 we use.

The solution is not unique!

Why does the Huffman's algorithm produce an optimal prefix code?

Correctness.

Theorem (Greedy property)

Let Σ be an alphabet, and x, y two symbols with the lowest frequency. Then, there is an optimal prefix code in which the code for x and y have the same length and differ only in the last bit.

Proof.

For T optimal with a and b sibilings at max. depth. Assume $f(b) \leq f(a)$. Construct T' by exchanging x with a and y with b. As $f(x) \leq f(a)$ and $f(y) \leq f(b)$ then $B(T') \leq B(T)$. $\square$

Theorem (Optimal substructure)

Assume T' is an optimal prefix tree for $(\Sigma - \{x, y\}) \cup \{z\}$ where x, y are symbols with lowest frequency, and z has frequency $f(x) + f(y)$. The T obtained from T' by making x and y children of z is an optimal prefix tree for Σ.

Proof.

Let T_0 be any prefix tree for Σ. Must show $B(T) \leq B(T_0)$.
We only need to consider T_0 where x and y are siblings. Let T_0' be obtained by removing x, y from T_0. As T_0' is a prefix tree for $(\Sigma - \{x, y\}) \cup \{z\}$, then $B(T_0') \geq B(T')$.
Comparing T_0 with T_0' we get,
$B(T_0') + f(x) + f(y) = B(T_0)$ and $B(T') + f(x) + f(y) = B(T)$,
Putting together the three identities, we get $B(T) \leq B(T_0)$. $\square$

Optimality of Huffman

Huffman is optimal under assumptions:

- The compression is lossless, i.e. *uncompressing the compressed file yield the original file*.
- We must know the alphabet beforehand (characters, words, etc.)
- We must pre-compute the frequencies of symbols, i.e. read the data twice

For certain applications is very slow (on the size n of the input text)